

UNIVERSITÀ DEGLI STUDI DI PALERMO

DIPARTIMENTO DI MATEMATICA E INFORMATICA

Data, Algorithms and Machine Intelligence

Approaches based on RAG and Prompt Engineering for Named Entity Recognition on Legal Documents

Thesis Author

Giuseppe Termerissa

Supervisors

Giosuè Lo Bosco

Giovanni Pilato

2025/26

Contents

Introduction **5**

I State Of The Art 8

1 Natural Language Processing **9**

1.1 Approaches, history and evolution of NLP 11

1.2 Natural Language Processing tasks 13

1.2.1 Token classification 15

1.2.2 Text generation 16

1.3 Named Entity Recognition 16

2 Transformers **18**

2.1 Attention 21

2.1.1 Multi-Head Attention 22

2.2 Encoder 25

2.3 Decoder 27

2.4 Case studies 30

2.4.1 Autoencoding Transformers: BERT 31

2.4.2 Autoregressive Transformers: GPT 33

3 Large Language Models **37**

3.1 Language Models 38

3.2 Pretraining and Supervised Finetuning 40

3.3 Quantization 41

3.4 Prompt Engineering . . . 43
3.4.1 In-Context Learning . . . 44
3.5 Retrieval Augmented Generation . . . 45

4 Approaches for Named Entity Recognition 49
4.1 Autoencoding Transformer approach . . . 50
4.2 Autoregressive Transformer approach . . . 51

II The Proposed Approach 53

5 Methodology 54
5.1 Problem Statement . . . 54
5.2 Datasets . . . 55
5.2.1 Kessler Italian Named-entities Dataset (KIND) . . . 55
5.2.2 Juridical Sentences . . . 56
5.3 Baseline . . . 59
5.4 Proposed Method . . . 61
5.4.1 Data Format . . . 61
5.4.2 Retrieval System . . . 65
5.4.3 Guidelines Prompt . . . 67
5.4.4 Large Language Model . . . 69
5.5 Experimental Setup . . . 71
5.5.1 Technical Environment . . . 71
5.5.2 Validation Strategy . . . 72
5.5.3 Evaluation Strategy . . . 72
5.6 Results . . . 73
5.6.1 Baseline Evaluation . . . 73

5.6.2 Pipeline Evaluation on KIND dataset 75
5.6.3 Pipeline Evaluation on Juridical Sentences dataset 77
5.7 Discussion 79
5.7.1 Comparison against the Baseline 79
5.7.2 Impact of Retrieval on Performance 81
5.7.3 Effects of Model Scale 82
5.7.4 Limitations and Future Directions 82

Conclusions **83**

Bibliography **85**

Introduction

Automatic analysis of natural-language documents, and particularly of domain-specific texts such as legal documents, is widely recognized as a challenging task. This is especially true when paired with *Named Entity Recognition*, i.e. the task of identifying and annotating entities within text, which is broadly recognized as one of the hardest tasks in the field of Natural Language Processing. One of the primary reasons for this difficulty lies in the fact that most texts are not realized with machine readability in mind. Instead, they are designed to satisfy specific needs and to be interpreted by human experts. Consequently, these documents frequently exhibit heterogeneous formats both in style and in encoding; they possess ambiguous phrasing, long nested sentences, and frequent references to external documents. All of these characteristics significantly make working with these texts harder, even to the most performant and well-known natural language processing pipelines. This is especially true in the case of classic system, usually associated with pre-determined rules that hardly apply to documents that exhibit such arbitrary features.

This problem becomes even more difficult in the case of domain-specific corpora, where documents may rely on conventions pertaining to a certain field in order to express meaning. This makes the application of general rules hard or even impossible, and a new need arises: the need to understand *context*.

In recent years, Natural Language Processing has undergone many revolutions, mostly orbiting around the need to automatically process context, leading to what is currently considered a modern technical revolution: the *Transformer model*, the foundation for *Large Language Models*. Large Language Models and, more generally, Transformers, are able to process text on the basis of any provided context. This is a very powerful tool, as it finally allows us to create tools that automatically parse,

analyze, and extract information from any arbitrary document through its impressive language-understanding capabilities and its vast world knowledge.

Despite these immense benefits, LLMs and Transformers are not perfect, and ideal workflows with these tools require dealing with the new set of problems that naturally arose from such an innovation. Transformers and LLMs are complex tools that introduce new challenges, the most common one being the need to deal with a training process that is extremely computationally expensive (creating a need for Quantization techniques, see 3.3). In the case of LLMs, their design allows us to gracefully adapt its generation abilities to any task through *In-Context Learning* (more in section 3.4.1), which is a huge benefit compared to classical Machine Learning and Deep Learning models where fine-tuning is necessary in order to use a model on a new domain.

However, LLMs have to deal with an entirely new problem: *hallucinations*. Hallucinations can be broadly defined as grammatically correct sentences that at first glance appear to make sense, but that are however not grounded in reality. This is a tough problem, as hallucinations can be hard to detect for both automated systems and human experts, due to their convincing nature.

To help with this new problem, a new technology is built on top of modern Large Language Models: *Retrieval Augmented Generation* (RAG). Retrieval Augmented Generation mixes Information Retrieval with Natural Language Processing, creating a new technology that relies on the retrieval power of search engines and on the generation power of LLMs to build systems that are able to generate sentences on the basis of known facts retrieved from a given database.

In this case RAG is, together with LLMs and In-Context Learning, a vital piece of the puzzle: it is what allows us to effectively annotate entities with correct labels in a way that is both meaningful, measurable and explainable. These are strong benefits

that are not always found in modern Artificial Intelligence, and as such present fertile grounds for further innovation.

This thesis aims to demonstrate that the ability of LLMs to instantly adapt to new domains is a valuable resource that allows us to quickly deploy Named Entity Recognition systems without needing a full fine-tuning step, thus saving immense amounts of computational resources by using a mix of Prompt Engineering and LLM-backed Information Retrieval, defining a system that is able to perform Information Extraction while remaining grounded in reality.

Part I

State Of The Art

This part of the thesis presents an overview of the current State Of The Art for Named Entity Recognition. In order to do so, presenting the fundamental technology is vital to further discuss how to approach NER, especially from the point of view of more modern advancements that include the use of technologies such as Transformers, Retrieval Augmented Generation and most importantly for the use case this thesis Prompt Engineering. The last one has proven to be especially important in the making of this project, as it was found that many models are extremely sensitive to minuscule changes, requiring some work in order to obtain the best possible results.

The methodological part of this work requires the understanding found in the following sections in order to be fully appreciated, and as such it is the objective of this part to fully provide the knowledge necessary to understand the technology behind the implementation.

1 Natural Language Processing

Natural Language Processing (NLP) is a subfield of Computer Science and Artificial Intelligence that focuses on the automatic elaboration, analysis, and machine understanding of natural language text. Its primary objective is to enable computational systems to process, and generate human language in a way that is both meaningful and useful. NLP is an inherently multidisciplinary field, heavily connected to linguistics, cognitive science, information retrieval, statistics, and knowledge representation. As such, NLP possesses a broad and historically rich background, with its origins tracing back to the early days of computing, notably to Alan Turing's work and the publication of the Turing Test as a philosophical and practical criterion for machine intelligence [1].

At a high level, Natural Language Processing addresses the problem of designing algorithms capable of making decisions based on what is being communicated using natural language. This broad definition encompasses a wide range of tasks that vary significantly in complexity and scope: these tasks include, but are not limited to, text classification, sentiment analysis, named entity recognition, sentence similarity estimation, speech recognition, automatic summarization, text generation, question answering, and machine translation.

In addition to these core tasks, NLP intersects with other fundamental areas of Computer Science, particularly *Information Retrieval* (IR). This intersection gives rise to applications such as document indexing, text ranking, search engines, and recommendation systems, where large collections of textual data must be efficiently searched and filtered. More recently, this overlap has led to the development of advanced techniques such as *Retrieval Augmented Generation* (RAG), which combine generative language models with external knowledge sources to improve factual ac-

curacy and contextual grounding. These technologies play an increasingly important role in modern NLP systems, vital to design a system that is able to effectively perform Named Entity Recognition.

The majority of NLP tasks have benefited significantly from the recent rise of modern statistical Artificial Intelligence and data-driven methodologies: in particular, the availability of large-scale datasets and the growth of computational power have enabled the widespread adoption of Machine Learning and Deep Learning approaches. Contemporary NLP systems are typically built around models that learn directly from data, automatically extracting statistical patterns and representations that approximate the underlying probability distribution of language. These learned representations can then be leveraged to solve a variety of downstream tasks with minimal task-specific engineering.

This data-centric paradigm is especially well suited to natural language, which is an exceptionally dense and complex domain. Even simple sentences can exhibit a vast number of variations due to differences in style, syntax, conjugation, inflection, and cultural usage. Furthermore, meaning in language is often implicit, relying on shared background knowledge rather than explicit wording. As a result, handcrafted rules often prove insufficient to fully capture the richness of human language. Instead, having access to large, diverse, and high-quality datasets combined with powerful learning algorithms has emerged as one of the most effective ways to model linguistic complexity and variability.

The following sections provide a deep overview of Natural Language Processing, covering its evolution, tasks, approaches, and its transformation in the modern era of Large Language Models (LLMs).

1.1 Approaches, history and evolution of NLP

The history of Natural Language Processing, similarly to the history of Artificial Intelligence, is commonly described as a sequence of complete shifts in methodology, each driven by changes in theoretical assumptions, computational resources, and data availability. The evolution of NLP can be grossly separated into three main eras: *symbolic NLP*, *statistical NLP*, and modern *deep learning-based NLP*.

The earliest phase, known as **Symbolic NLP** (1950-1970), is rooted in formal reasoning, where linguistic competence is encoded explicitly through hand-crafted rules. These rules are derived from linguistic theories and aim to model grammar, syntax, and semantics. Systems developed under this approach rely on predefined vocabularies, context-free or context-sensitive grammars, and deterministic parsing strategies. Importantly, *symbolic NLP does not attempt to model language statistically or probabilistically*: instead, it assumes that linguistic knowledge can be exhaustively specified by human experts which define sets of rules for the system. While these systems were successful in constrained and well-defined domains, they quickly revealed severe limitations when confronted with real-world language, which is inherently ambiguous and context-dependent, resulting in systems that are both hard to design and hard to scale.

The second major phase is **Statistical NLP** (1980-1990), which marked a fundamental shift in how language was modeled. Rather than relying on explicitly defined linguistic rules, statistical NLP borrows from Machine Learning, learning patterns directly from data. In this paradigm, language is learned by estimating probability distributions over words, sequences, or linguistic structures. Classic models from this era include n-gram language models, Hidden Markov Models (HMMs), and probabilistic context-free grammars. The n-gram model in particular played a central

role by approximating language as sequences of n tokens, enabling applications such as next-word prediction, speech recognition, and early machine translation systems. Although statistical methods significantly improved robustness and adaptability compared to symbolic systems, they were still constrained by sparse data, limited context windows, and the need for extensive feature engineering.

The current phase, often referred to as **Deep Learning NLP** (2010-present), represents a further paradigm shift driven by advances in neural networks, optimization techniques, and large-scale computing infrastructure. In this era, NLP systems are built around deep neural architectures capable of learning hierarchical representations of language directly from raw text. Early successes involved recurrent neural networks (RNNs) and convolutional neural networks (CNNs). However, these were soon surpassed by attention-based architectures. As discussed in Section 2, the most influential and effective model to emerge in this context is the Transformer architecture, originally introduced for machine translation in the landmark paper "Attention Is All You Need" [26]. Since their introduction in 2017, Transformers have rapidly become the dominant architecture across virtually all NLP tasks. They have demonstrated state-of-the-art performance in text classification and representation learning through encoder-based models such as BERT [9], as well as in text generation through decoder-based such as GPT [17]. One of the most important properties of large Transformer-based language models is their ability to generalize beyond the tasks they were explicitly trained on, through mechanisms such as few-shot learning [31] and in-context learning. Transformers are deeply analyzed in section 2.

Transformers have profoundly reshaped the NLP landscape, most notably by introducing conversational agents such as ChatGPT. These systems have demonstrated unprecedented levels of natural language understanding, leading to widespread adoption across both academic and industrial domains. From a commercial perspective,

the impact has been equally significant, with platforms based on large language models reportedly generating tens of millions of dollars in revenue within a few years of their introduction [2].

1.2 Natural Language Processing tasks

The purpose of this section is to outline some of the most relevant NLP use cases, highlighting how different methods and models are employed to address fundamental problems involving natural language.

- **Information Retrieval** refers to the problem of identifying and retrieving a set of documents that are relevant to a user's query. This task lies at the core of search engines and typically involves ranking large collections of text according to their semantic relevance to a given input.

- **Token Classification** is the task of assigning a label to each individual token within a sentence or document. This formulation is used in a variety of applications, including part-of-speech tagging and named entity recognition, where fine-grained, token-level understanding is required.

- **Text Generation** concerns the automatic production of coherent and grammatically correct natural language text. Depending on the application, this may involve generating responses or summaries, often conditioned on an input prompt or contextual information.

- **Named Entity Recognition** is a specific instance of token classification focused on identifying and categorizing meaningful entities within text, such as people, organizations, locations, dates, and domain-specific entities. This task

plays a crucial role in information extraction pipelines and downstream reasoning systems.

- **Text Classification** involves assigning one or more labels from a predefined set to an entire text segment, such as a sentence or a document. Common applications include sentiment analysis or spam detection.

- **Question Answering** addresses the problem of automatically producing an answer to a question posed in natural language. Depending on the system, answers may be generated freely by a model or derived through a combination of retrieval and reasoning.

- **Summarization** focuses on reducing the length of a given text while preserving its most important semantic content.

- **Machine Translation** is the task of translating a sentence or document from one language to another while preserving its meaning, style, and grammatical correctness. This has historically been one of the driving forces behind many advances in NLP research.

- **Speech Recognition** involves converting spoken language into written text.

While all these tasks share natural language as their common denominator, they represent fundamentally different computational problems, each with their own assumptions, challenges, and solution strategies. Consequently, they often require distinct modeling approaches, evaluation metrics and datasets.

This document primarily focuses on *token classification* and *text generation*, each possessing the ultimate goal of being applied to Named Entity Recognition for legal documents. The discussion of these tasks aims to provide the conceptual and technical

foundations necessary to understand the methods and systems introduced in the later sections of this work. *Information Retrieval* is also attentioned later in the thesis in section 3.5.

1.2.1 Token classification

Token classification is the task of assigning a label to each individual token in a sequence, where labels are drawn from a predefined and finite set. Token classification includes several subproblems, including Part-of-Speech tagging and, most notably, Named Entity Recognition (NER). In all these cases, the goal is to associate each token with a category that reflects its role or meaning within the context of the sentence.

Classical approaches to token classification relied on feature-based machine learning methods. Models such as Hidden Markov Models, Conditional Random Fields and Support Vector Machines [19] were widely adopted, often in combination with manually engineered features capturing contextual information. A typical objective would be to learn the decision boundaries that model the association between tokens, often approximating the underlying probability distribution of these associations.

Modern solutions to token classification predominantly rely on deep learning architectures and in particular on Transformer-based models. By leveraging contextualized word representations, these models can capture dependencies and contextual cues that are crucial for accurate token-level predictions. A typical real-world use case for token classification is the automatic extraction of structured information such as identifying dates, amounts, and entities from unstructured, complex documents.

Token classification datasets are typically constructed by associating each token in a text sequence with a corresponding label. In many cases, additional contextual information is implicitly provided through the surrounding tokens, as is particularly

important for tasks such as Named Entity Recognition, where the correct classification of a token often depends on its role within a multi-token span or the broader sentence context.

1.2.2 Text generation

Text generation refers to the task of automatically producing coherent and contextually appropriate natural language text, often conditioned on an input prompt or an initial piece of text. At its core, the objective is to generate sequences of tokens that form grammatically and semantically correct meaningful sentences.

Beyond its standalone applications, text generation has increasingly been used as a unifying paradigm to address other NLP tasks. Modern Large Language Models can reformulate classification, summarization, translation, and question answering as generative problems, producing outputs directly in natural language rather than selecting from predefined label sets. As discussed in Section 5.4.4, this generative capability enables models to perform tasks they were not explicitly trained for, often using only a small number of examples or natural language instructions. Consequently, text generation has become a central component of contemporary NLP systems and a key driver of recent advances in the field.

1.3 Named Entity Recognition

Named Entity Recognition (NER) is a core task in Natural Language Processing whose objective is to identify and classify mentions of real-world entities within unstructured text. Given an input sequence, the task consists of two subtasks: (1) detecting spans of tokens that correspond to entities and (2) assigning each detected span a category drawn from a label set, such as PER (Person), LOC (Location),

ORG (Organization), MISC (Miscellaneous) or more domain-specific classes. NER is widely regarded as one of the most challenging NLP problems, primarily due to the intrinsic ambiguity of natural language and the strong dependence on contextual information.

Over the years NER has been approached from multiple methodological perspectives. Early systems relied heavily on handcrafted rules and regular expressions. While effective in constrained domains, such approaches proved difficult to scale. More recently, deep learning methods have become dominant, achieving state-of-the-art performance across most benchmarks and forming the methodological backbone of this work.

In addition to methodological diversity, NER has also been characterized by a variety of annotation schemes. Common labeling formats include IO, BIO, and BIOES, each encoding entity boundaries in a slightly different way. For instance, the BIO scheme explicitly marks the beginning (B) and inside (I) of an entity span, while BIOES further distinguishes single-token entities (S) and end tokens (E). The choice of annotation format has implications for model design, and even though it is often dictated by dataset availability, it must be accounted for when running experiments.

As shown in [21], NER research has evolved through several distinct phases, each introducing new modeling assumptions and technical innovations. These include:

- **Rule-based NER (2000)**: systems leveraging POS tagging, lexical patterns, and handcrafted linguistic rules;
- **Supervised NER (2006)**: statistical models such as SVMs, CRFs, LSTMs, and Bi-LSTMs trained on annotated corpora;
- **Semi-supervised NER (2007)**: techniques incorporating self-training and adversarial learning to exploit unlabeled data;

- **Nested NER (2009)**: region-based and generation-based methods designed to handle overlapping or hierarchical entities;
- **Relation Extraction (2016)**: span-based formulations that jointly model entities and their relations;
- **Low-resource NER (2016)**: approaches leveraging external knowledge bases, transfer learning, and in-context learning;
- **Prompt-based NER (2018)**: reformulations of NER as a prompting or generation task using pretrained language models;
- **Continual learning (2022)**: techniques such as knowledge distillation to incrementally adapt models without catastrophic forgetting.

The remainder of this work focuses on modern NER approaches that rely on transformer-based architectures, combining supervised learning and recent advances in Large Language Models. In particular, special attention is given to methods that exploit in-context learning, external knowledge sources, and Retrieval Augmented Generation techniques to enhance robustness and adaptability in real-world scenarios [23].

2 Transformers

At the core of this thesis lies a specific class of deep learning architectures that has profoundly reshaped the field of Natural Language Processing: the Transformer. Transformer models are neural network architectures originally introduced by Vaswani et al. in "Attention Is All You Need" [26]. Unlike previous models such as RNNs, Transformers leave recurrence behind entirely and instead rely on a mechanism known as

attention to model relationships between tokens. In their original formulation, Transformer models perform inference through the interaction of two major components: the encoder and the decoder.

The *encoder* is responsible for processing the sequence of raw input text. After tokenization and embedding, the input tokens are transformed into a sequence of dense, context-aware vector representations. Through multiple stacked layers of self-attention and feed-forward networks, the encoder captures both local and long-range dependencies (more in section 2.2). On the other hand, the *decoder* is designed to generate output sequences: it operates autoregressively, meaning that each generated token takes into account both the previously generated tokens and, when present, the representations produced by the encoder. During generation, the decoder uses masked self-attention to prevent access to future tokens, ensuring that predictions respect the causal structure of language. More on decoders in section 2.3.

Although the original Transformer architecture combines both encoder and decoder components, these modules can also be employed independently. Encoder-only models, often referred to as *autoencoding Transformers*, form the basis of architectures such as BERT and its variants. These models are typically trained using masked language modeling objectives and are particularly effective for tasks that require an understanding of context, including text classification, token classification, and semantic similarity estimation. By contrast, decoder-only models (also known as *autoregressive models*) form the backbone of many modern conversational agents and large-scale language platforms, such as ChatGPT, Claude, and DeepSeek. However, the raw pretrained models are usually not sufficient for safe and effective real-world deployment. Additional training stages, such as Reinforcement Learning from Human Feedback (RLHF), are typically applied to align model behavior with human expectations.

The defining characteristic of Transformer architectures is the concept of **attention**. Attention mechanisms enable the model to assign different levels of importance to different tokens within a sequence, effectively learning which parts of the input are most relevant for a given prediction. By computing weighted combinations of token representations, attention allows the model to focus selectively on informative elements of the text while giving less importance to less relevant ones. This capability is central to the Transformer's success, as it provides a flexible and powerful means of modeling contextual relationships without relying on fixed positional constraints.

Thanks to attention, Transformers are able to capture complex linguistic phenomena such as long-distance dependencies, making them highly versatile and applicable to a wide range of NLP tasks. Common applications include classification tasks such as sentiment analysis, sequence labeling tasks such as token classification and Named Entity Recognition, and information extraction tasks in specialized domains.

In recent years, the focus has shifted toward the generative capabilities of Transformer models. As explored in later sections, these generative properties mainly arise from the decoder component and have enabled the development of systems capable of reasoning, summarization and dialogue.

For all of the above reasons, Transformers constitute the architectural backbone of the systems investigated in this thesis and serve as the primary tool for exploring automatic document analysis in challenging, real-world domains.

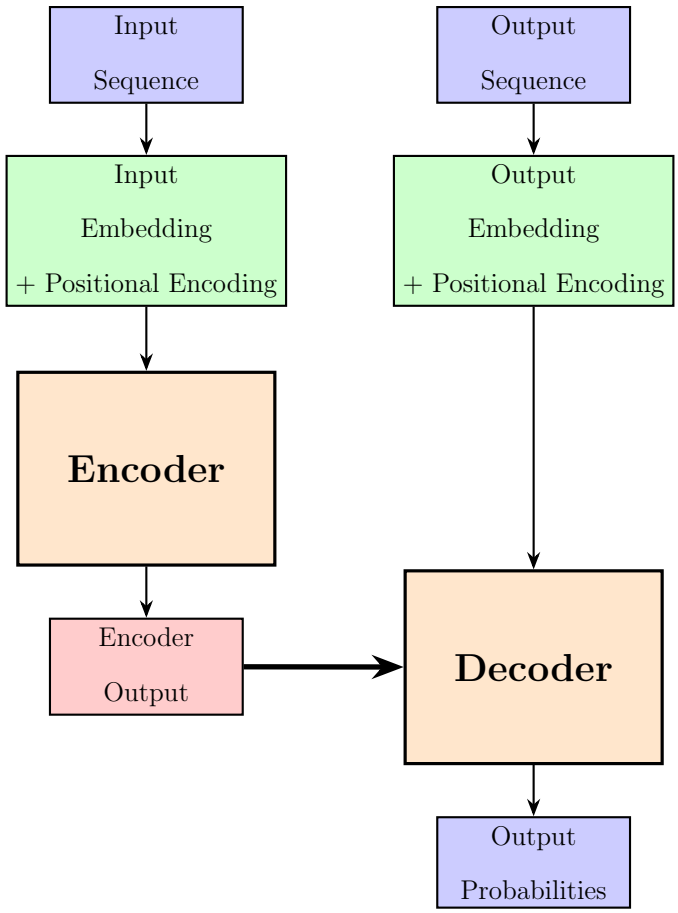


Figure 1: General overview of the transformer architecture

2.1 Attention

Attention is the mechanism that allows transformer models to focus on the most relevant parts of any given input sequence. This is achieved by calculating a set of attention scores for each token, each of which determines the importance of each word in a sentence.

In Vaswani et al. [26], attention is defined through a mechanism known as **self-attention**, implemented through multi-head self-attention layers. This variant of attention allows every token to interact with every other token in order to determine

its level of importance.

Given queries q, keys k and values v, the model is going to look for tokens in k that are most similar to the ones present in q. At this point, its value v is retrieved. This mechanism can be succinctly expressed as

$$\text{self-attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \tag{1}$$

where Q, K and V are the matrical forms of the above mentioned arrays.

This form is used in standard implementations of attention as it allows for efficient parallel processing through the use of GPUs, highly prevalent in the field and especially useful in the current use case.

As mentioned above, every token interacts with each other in self-attention: this induces a complexity of $O(n^2)$ where n is the sequence length. Because of this, alternative implementation exist that provide a series of optimizations such as Longformer [6], which implements sparse attention, and Sliding Window Attention [10], which implements local attention.

2.1.1 Multi-Head Attention

Multi-Head Attention is a core component of Transformer architectures and represents one of the innovations that distinguish them from earlier sequence modeling approaches. Rather than computing a single attention distribution over the input sequence, Transformers employ multiple attention heads in parallel, each of which is able to attend to different aspects of the input. This design allows for higher resolution, allowing the model to capture diverse relational patterns within the same sequence and to represent information at multiple levels of abstraction at the same time.

As highlighted in the original Transformer paper [26], recurrent neural networks suffer from a fundamental limitation: *the number of operations required to relate two arbitrary positions in a sequence grows with the distance between them.* This makes learning long-range dependencies particularly challenging, especially in long texts. In contrast, the self-attention mechanism used in Transformers reduces this cost to a constant number of operations, enabling any token to directly attend to any other token in the sequence. However, this efficiency comes at the cost of reduced effective resolution, since attention involves weighted averaging over token representations. Multi-Head Attention is introduced precisely to counteract this effect by allowing the model to attend to multiple representation subspaces in parallel.

Intuitively, each attention head can be understood as focusing on a different "view" of the input sequence, analogous to how different kernels in a Convolutional Neural Network extract different types of features from the same input. Some heads may specialize in capturing syntactic relationships, while others may focus on semantic dependencies. The outputs of all heads are then concatenated and linearly transformed, resulting in a richer and more expressive representation than could be obtained with a single attention operation. As a consequence, the number of attention heads becomes an important architectural hyperparameter, directly influencing the model's representational capacity and computational cost.

Beyond the original formulation, several optimizations and variants of Multi-Head Attention have been proposed to improve efficiency. The main optimization concerns are about how the query Q, key K, and value V matrices are computed and shared across attention heads. Different design choices in this regard lead to distinct attention mechanisms, each with its own trade-offs between expressiveness and performance:

- In classical **Multi-Head Attention** (MHA), separate linear projections are used to compute the Q, K, and V matrices for each attention head. This maximizes flexibility, as each head can learn entirely independent representations, but it also incurs higher memory and computational costs.

- In **Grouped-Query Attention** (GQA) [5], multiple query heads are grouped to share the same key-value pairs. This approach reduces the number of key-value projections while preserving much of the diversity introduced by having multiple query heads, striking a balance between efficiency and representational power.

- In **Multi-Query Attention** (MQA) [22], all attention heads share the same key and value matrices, while maintaining separate queries. This significantly reduces memory usage and accelerates decoding, making MQA particularly attractive for autoregressive generation in large language models.

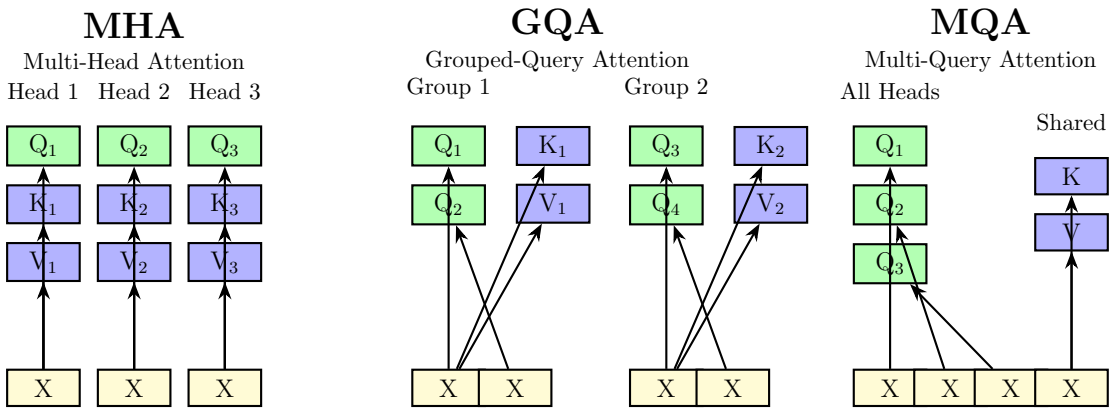


Figure 2: Comparison between different self-attention mechanisms

These attention variants have become increasingly important as Transformer models scale to billions of parameters; as such, carefully choosing which variant to use has become of increasing importance in recent years.

2.2 Encoder

The encoder is the component of the Transformer architecture whose primary objective is to process an input sequence and produce a rich, contextualized representation of each token. In its canonical formulation, the encoder is trained to predict missing or masked tokens given their surrounding context, a strategy that underlies many encoder-only models such as BERT. More generally, the encoder learns to transform raw input text into dense vector representations that preserve semantic meaning, syntactic structure, and relational information across the entire sequence.

Architecturally, the encoder is composed of N identical blocks stacked sequentially, where N is a hyperparameter that controls the depth of the model. Each block follows the same structural pattern and consists of two main sublayers: a *multi-head self-attention layer* and a *feed-forward neural* network. Through repeated application of these blocks, the encoder progressively refines token representations, allowing information to flow across the sequence and enabling each token to incorporate context from all other tokens.

The encoding process is designed to ensure that meaning, relationships between words, and properties of the input are preserved and enriched throughout the network. This is particularly important for tasks that require deep language understanding, such as token classification, semantic similarity estimation, and information extraction.

To achieve these results, the encoder relies on several fundamental building blocks that are applied in sequence:

1. **Input Embedding**, which maps each token in the input sequence to a dense vector representation. These embeddings capture semantic properties of words and provide the initial numerical representation upon which the model operates.

2. **Positional Encoding**, which injects information about the position of each token within the sequence. Since the self-attention mechanism itself is invariant to token order, positional information is incorporated through positional embeddings, allowing the model to account for order, distinguish between tokens appearing in different positions and to reflect the intuition that nearby tokens are often more strongly related.

3. **Multi-Head Self-Attention Layer**, which enables each token to attend to all other tokens in the sequence, weighting their contributions according to relevance. This layer implements the attention mechanisms described in Section 2.1 and is responsible for capturing contextual dependencies across the input.

4. **Feed-Forward Layer**, a fully connected neural network applied independently to each token representation. This layer further transforms the output of the attention mechanism, allowing the model to extract higher-level features and introduce additional non-linearity.

In addition to these core components, each encoder block incorporates residual connections around both the attention and feed-forward sublayers, which helps to stabilize training and facilitate gradient flow in deep networks, allowing the model to learn incremental refinements over earlier representations. Layer normalization is applied after each sublayer to improve convergence and ensure numerical stability during training.

The combination of self-attention, feed-forward transformations, residual connections, and normalization enables the encoder to build expressive and robust representations of input text. The original encoder architecture, as introduced in the Transformer model, is illustrated in Figure 2.2.

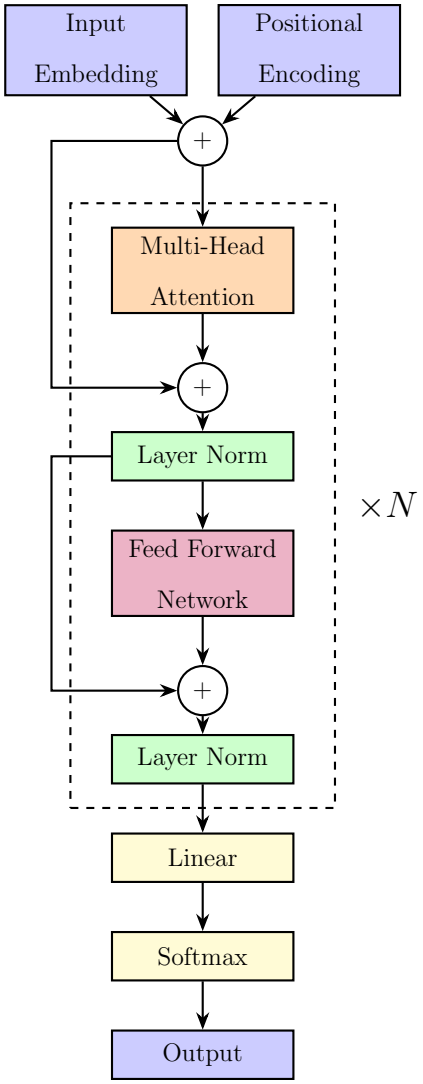


Figure 3: Original encoder architecture

2.3 Decoder

The decoder is the central component of the transformer architecture responsible for generating the output sequence, which in natural language processing tasks corresponds to the predicted words or tokens. It utilizes multi-headed attention to capture long-range dependencies within the input sequence, with a crucial difference compared

to the encoder: while encoders look forward in the sequence, decoders work in an autoregressive manner, masked by using special tokens. This limitation ensures that the model can only access and analyze the preceding tokens in the sequence, creating a dependency on the preceding context for predicting the next token.

The original decoder architecture, depicted in Figure 2.3, highlights the core elements that enable the decoder to generate the output sequence. The masking process, coupled with the multi-headed attention mechanism, allows the model to capture the long-range dependencies within the input sequence while maintaining the autoregressive property. This property ensures that the model can accurately predict the next token in the sequence based on the preceding context, ultimately leading to the generation of meaningful and coherent output.

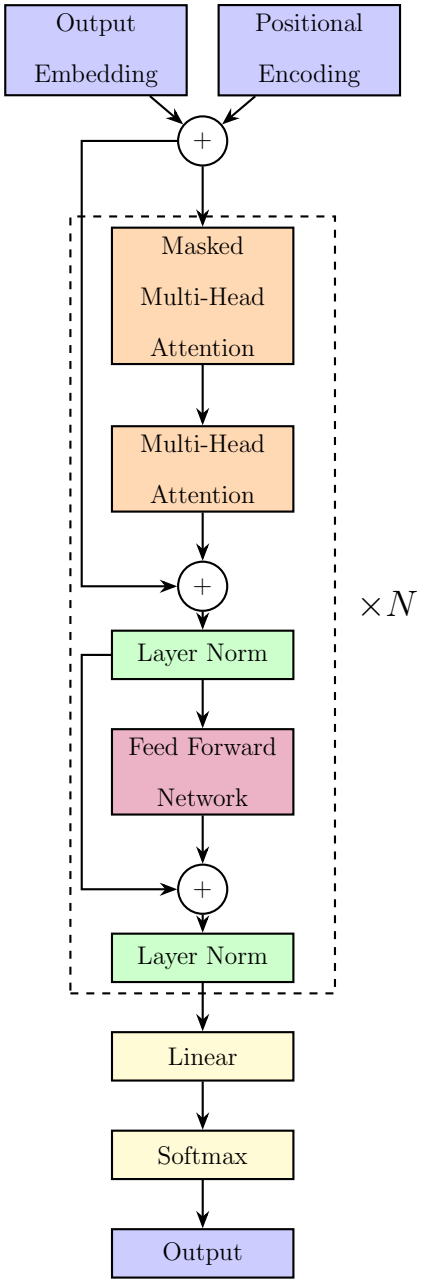


Figure 4: Original decoder architecture

2.4 Case studies

Even though transformers were originally designed for translation, the modern idea of transformers mainly orbits around that of generative AI and, specifically in this context, text generation. Despite this fact, transformers are highly versatile tools that are actually capable of adapting to various different tasks as long as the architecture is properly designed, which is all but an easy task. This is especially true when other factors come into play, such as the ones introduced by prompt engineering, as shown in section 3.4.

As previously mentioned throughout this work, transformers can attend to different tasks by specializing the model on either one or both of the components, inducing three sub-architectures:

- **autoencoding transformers**, specialized in performing tasks such as classification and summarization;
- **autoregressive transformers**, specialized in text generation;
- **sequence-to-sequence transformers**, specialized in problems where there is a length mismatch between input and output (e.g. in Machine Translation).

Additionally, transformers may be further specialized in one area through other means such as Reinforcement Learning, known for being the de-facto technique for creating chatbots.

The objective of this section is to investigate the differences in architecture between these variations, providing a detailed writeup of the inner workings of each variant.

2.4.1 Autoencoding Transformers: BERT

BERT is highly praised for its capacity in solving non-generative tasks such as text classification, question answering and summarization. **BERT (Bidirectional Encoder Representation from Transformers)** [9] is a language model based on autoencoding transformers that presents an innovative approach to language representation learning, which propelled its success in various NLP tasks like classification, question-answering, and summarization.

BERT employs a stacked encoder design, where each encoder layer captures contextual information from the input sequence. The process is bidirectional, meaning the model attends to the words both before and after the current word, improving the representation with richer context. Since this model is known as an autoencoding transformer, it is a deep neural network that relies on stacking N encoders defined as in section 2.2.

The base working mechanism is to predict special missing tokens using a given context, with the objective of encoding natural language in such a way that:

- the *meaning* of each word is encoded, given the full context of the sequence;
- the *relationships* between different words in the sequence is encoded;
- the *syntactic and structural information* of the input is encoded.

Pretraining in BERT consists in analyzing vast amounts of text data to train on two objectives:

- **Masked Language Modeling** (MLM): Predicts the missing word in a sequence, encouraging the model to understand the meaning of words in context.

- **Next Sentence Prediction** (NSP): Determines if two sentences are logically related, improving the ability of the model to capture semantic relationships between words.

The Masked Language Modeling (MLM) task is what causes BERT to interpret input sequences bidirectionally. This is simply because the model looks to the left and right context with respect to the missing token to try and predict it. Next Sentence Prediction (NSP), despite its name, is not a text generation task: instead, it is a binary classification problem, outputting 0 if B does not follow A and 1 otherwise. BERT pretraining datasets must account for this objective by providing this value properly for each pair of sentences.

In practice, each input is a pair of concatenated sentences with some words being masked. BERT combines three types of embeddings to represent words: token, position and segment embeddings. In detail:

- **Token Embeddings**: Represent tokens as embeddings that semantically capture a given word (or part-of a word).
- **Position Embeddings**: Encode the position of each word in the sequence, capturing information related to the ordering of words.
- **Segment Embeddings**: Represent the sentence index (sentence 1 or sentence 2), helping the model capture sentence-level context.

BERT then outputs two values: a **Binary Output** that indicates whether two sentences are related or not, produced by the Next Sentence Prediction task and a **Word Vector Output**, representing the word in the context of the entire sentence and produced by the Masked Language Modeling task.

Finally, the word vectors are passed to a softmax layer with 30,000 neurons and compared to the ground truth by using a loss function, such as Cross Entropy loss.

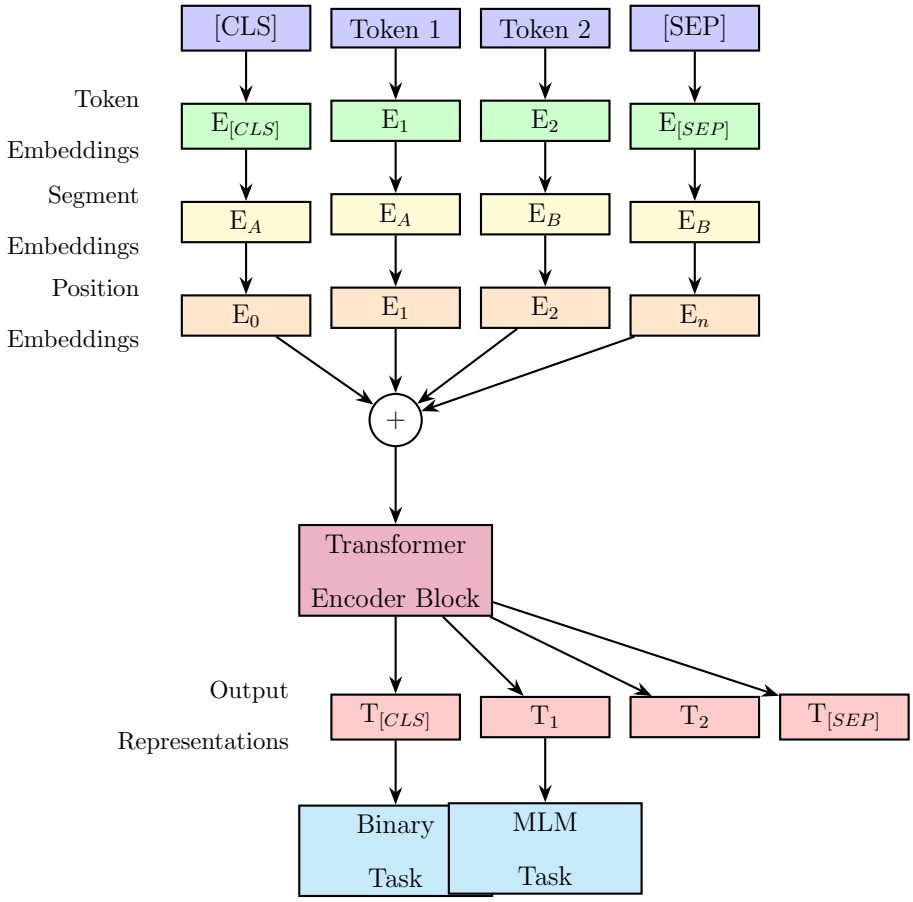


Figure 5: BERT architecture

2.4.2 Autoregressive Transformers: GPT

Autoregressive transformers are language models that are built exclusively using stacks of decoder blocks, as described in Section 2.3, and are designed to predict the next token in a sequence given all previously observed tokens. By construction, this formulation directly models natural language as a sequence, making autoregressive Transformers particularly well suited for text generation.

During pretraining, the next inputs are masked and the model can only look at

tokens before the attention head: the model is rewarded for successfully predicting the next token based on the already known ones, hence obtaining the autoregressive property. Models that implement this architecture include GPT, Llama and DeepSeek-R1, among others.

GPT (Generative Pre-Training) in particular is the original model based on autoregressive transformers, which paved the way for this class of models since its introduction in June 2018 [17]. Pretraining on GPT is performed in a self-supervised fashion, similarly to the n-gram model briefly seen in section 1.1.

The training procedure as seen in the original paper of GPT, "Improving Language Understanding by Generative Pre-Training", consists of two stages: **unsupervised pre-training** and **supervised fine-tuning**. Unsupervised pre-training consists of using an unsupervised corpus of tokens $\mathcal{U}$, used as a *standard language modeling objective*:

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, ..., u_{i-1}; \Theta) \tag{2}$$

Then, *supervised fine-tuning* is performed. Assuming a labeled dataset $\mathcal{C}$, the inputs are passed through the model to obtain (1) the activation of the final transformer block h_l^m, which is then fed into (2) a linear output layer to predict y:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, ..., x^m) \tag{3}$$

Finally, it has been found that including language modeling as an auxiliary objective further helped with finetuning. For this reason, the objective is modified as:

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda L_1(\mathcal{C}) \tag{4}$$

where λ is a weight in $\mathbb{R}$.

Architecturally, GPT shares similarities to BERT, presenting a self-attention layer followed by a feed forward layer. Differently from BERT and autoencoding transformers in general though, the input is composed of text and position embeddings rather than raw text, and the output consists of predicted tokens.

The biggest hurdle when pretraining GPT is that a large amount of data is needed in order to train it [17]. For this reason, techniques known as *meta-learning* have been adopted in order to efficiently finetune the model to a given task without needing explicit training. A very notable example of meta-learning is *in-context learning* (see section 3.4.1), extremely useful to adapt a given model to any given task when used in the form of prompt engineering.

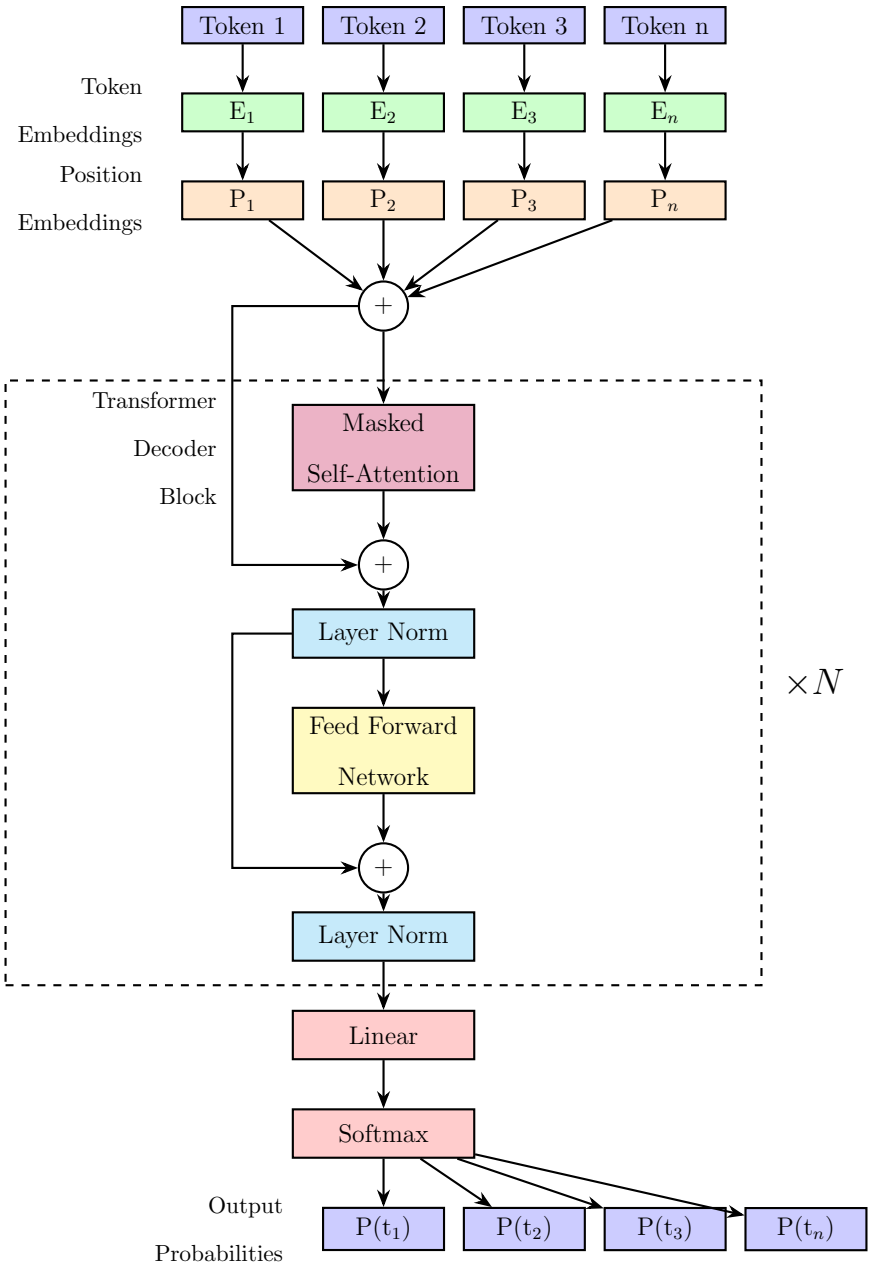


Figure 6: GPT architecture

3 Large Language Models

One specific piece of technology lies at the very core of the current renaissance in Artificial Intelligence: **Large Language Models** (LLMs). LLMs are language models based on neural networks [16] and more specifically on the Transformer architecture. These models typically contain hundreds of billions of trainable parameters and are trained on massive corpora of text. The objective of this large-scale training is to produce models that are capable of generating fluent and contextually appropriate natural language while also exhibiting the ability to perform a wide array of tasks based solely on natural language instructions.

An important characteristic of Large Language Models is that many of their most impressive capabilities are not explicitly given through supervised training, but instead emerge as a consequence of its scale: as model size, training data, and computational resources increase, LLMs begin to display *emergent behavior*, such as in-context learning and few-shot generalization, granting the ability to adapt to unseen tasks without task-specific fine-tuning.

From an architectural standpoint, this definition implies that Large Language Models are almost exclusively built using autoregressive Transformer architectures. In practice, all LLMs rely on the decoder component of the Transformer, trained with a causal language modeling objective to predict the next token in a sequence given all preceding tokens. Autoregressive decoding provides a simple yet powerful training signal that scales naturally with data size and enables text generation across a wide variety of contexts and domains. As a matter of fact, models such as GPT [17], LLaMA [25], Gemma [24], and DeepSeek-R1 [11] are all decoder-only Transformer models. Despite differences in size, training data and optimization strategies, these models share a common architectural foundation centered on autoregressive

generation.

The reliance on next-token prediction as a training objective has proven to be remarkably effective. By optimizing this single objective on a vast and diverse corpora of text, LLMs implicitly acquire knowledge about grammar, semantics, discourse structure and even world knowledge, encoded in text.

At the same time, the immense scale and generality of Large Language Models introduce new challenges: issues related to computational cost, energy consumption, hallucinations, and interpretability have become massive concerns in recent years. As a result, significant efforts have been devoted to developing techniques such as pretraining, prompt engineering and quantization, among others, all of which are explored in later sections and whose effects are explored in the methodological part of this work.

3.1 Language Models

Large Language Models are a direct evolution of classical language modeling, a task in Natural Language Processing that aims to establish a computational framework for understanding and generating language. Language modeling seeks to estimate the likelihood of a sequence of words or tokens and, more specifically, to predict the next word given a preceding context.

This objective induces the model into implicitly capturing aspects of natural language such as syntax and meaning, all in a self-supervised fashion. What this means is that rather than relying on manually annotated data, models are trained by automatically inspecting large text corpora. For instance, a model may be trained to predict the next token in a sequence, using the surrounding text as context. Through this process, the system automatically extracts patterns from raw language data,

which then serve as training examples. Over time, the model learns to approximate the underlying probability distribution of the text corpus, all without direct human intervention in the labeling process.

Historically, both probabilistic and neural approaches to language modeling have converged around two main training paradigms, which differ in how context is defined and how predictions are formulated:

1. **Causal Language Modeling**, which estimates the probability of the next token given all previous tokens in the sequence. This formulation is used by both classical n-gram models as well as modern autoregressive architectures such as GPT. It naturally supports text generation, as tokens can be produced sequentially by repeatedly sampling from the model's distributions.

2. **Masked Language Modeling**, which estimates the probability of one or more masked tokens given the remaining visible tokens in the context. This bidirectional formulation is characteristic of models such as BERT and is particularly effective for understanding-oriented tasks that do not require generation.

Training effective language models requires vast amounts of textual data to properly capture the complexity of natural language. Additionally, the computational cost of training large neural networks over such datasets is massive, often requiring hours of computation in order to properly setup a model for a task. These constraints have strongly influenced the development of modern training strategies in the field.

To address these challenges, a very common paradigm has emerged: *pretraining* followed by *fine-tuning*. In this framework, a model is first pretrained from zero on a general-purpose corpus using a self-supervised language modeling objective, such as the one described above. The resulting model acquires broad linguistic knowledge that is not tied to any specific task. Once pretrained has finished, this pretrained

model is fine-tuned on smaller, task-specific datasets, allowing it to adapt its outputs and internal representations to the requirements of a particular application. Fine-tuning may involve supervised learning, instruction-based objectives, or reinforcement learning, depending on the task and deployment scenario.

The combination of large-scale pretraining and targeted fine-tuning forms the foundation of *transfer learning* in Large Language Models. By reusing knowledge acquired during pretraining, models can achieve strong performance on tasks even when labeled data is limited. As discussed in the following sections, this transfer learning paradigm is the key to the flexibility of modern Large Language Models.

3.2 Pretraining and Supervised Finetuning

As mentioned earlier, *pretraining* is the starting point of any modern LLM: it consists of an extensive training phase that is performed on a model that is being trained for the first time. Typically, a model undergoing pretraining is trained on large and general-purpose text corpora that span a wide range of topics and writing styles. By learning from such diverse data, the model acquires broad linguistic knowledge as well as a substantial amount of implicit world knowledge encoded in text. The resulting pretrained model is not specialized for any single task, but instead serves as a versatile foundation capable of supporting many downstream applications.

Supervised Finetuning (SFT) refers to the process of modifying a pretrained model's behavior by further adjusting its weights using labeled data. The primary goal of SFT is to adapt and align a general-purpose language model to specific tasks, domains, or user requirements. While pretraining equips the model with broad linguistic and world knowledge, SFT specializes this knowledge by teaching the model how to produce desired outputs for well-defined inputs.

In contrast to pretraining, SFT is typically performed on curated datasets. These datasets may consist of pairs of prompts and corresponding responses, task-specific annotations, or structured examples reflecting the target behavior. Because the model already possesses strong language understanding capabilities, SFT generally requires significantly fewer examples than pretraining, often ranging from thousands to a few million samples, and as a consequence is often much easier on computational resources.

From an optimization perspective, SFT uses standard supervised learning objectives, such as cross-entropy loss, to encourage the model to generate outputs that closely match the provided labels.

This two-stage process composes *transfer learning* for LLMs. It is an extremely important piece of modern Artificial Intelligence as it significantly reduces the amount of labeled data required for tasks, allowing models to be adapted to new tasks as long as a suitable corpora of text can be identified.

Notably, *Parameter-Efficient Fine-Tuning* (PEFT) is a class of fine-tuning strategies designed to adapt large transformer models while updating only a small subset of their parameters, achieving performance comparable to full fine-tuning at a fraction of the computational and memory cost. This is achieved by freezing the original weights of the model and introducing a limited number of trainable parameters, known as *adapters*.

3.3 Quantization

One of the most significant constraints during the deployment of Large Language Models is memory availability. This limitation is particularly problematic in consumer settings, where resources are typically limited. GPUs rely on dedicated memory (VRAM) which is physically separate from the system's main RAM and optimized

for massively parallel computation, and large amounts of VRAM is usually needed in order to run these models, increasing with the number of parameters. While VRAM offers faster access times compared to CPU memory, it is usually available in much smaller quantities. Because of all of this, memory quickly becomes a significant bottleneck in the employment of LLMs.

To address this issue, *quantization* techniques have been developed with the goal of reducing the memory footprint of neural networks. Quantization consists of converting model parameters from high-precision numerical representations, such as 32-bit floating point (FP32), to lower-precision formats such as FP16, INT8, or even INT4. By reducing the number of bits required to represent each parameter, quantization directly decreases VRAM usage and often improves inference speed.

As shown in recent studies [13, 8], carefully designed quantization methods can achieve substantial memory savings while preserving most of the model's original performance. In particular, INT8 and INT4 quantization schemes have proven particularly effective for inference.

Building upon these ideas, *QLoRA* (Quantized Low-Rank Adaptation) represents a further step toward memory-efficient fine-tuning and deployment. QLoRA combines aggressive quantization of the base model with Parameter-Efficient Fine-Tuning techniques such as Low-Rank Adaptation (LoRA) [12]. In this setup, the pretrained model weights are stored in quantized form and kept frozen, while low-rank adapter matrices remain in higher precision and are trained during fine-tuning. This hybrid approach allows models with a high number of parameters to be fine-tuned on a single GPU without significant performance degradation.

3.4 Prompt Engineering

Alongside the recent developments of Large Language Models, a new subfield has emerged that directly engages with every aspect of these novel systems and that is so all-encompassing to computing in general to the extent of creating entirely new professional roles [27]: *prompt engineering*.

In general terms, a *prompt* is the input provided to a language model that conditions and guides its output, shaping both the content and the style of the generated response [20]. Prompts may take a wide variety of forms, ranging from minimal instructions to highly structured and detailed specifications that include explicit constraints, examples, formatting rules. For example, the latter is the approach chosen to guide the LLM into generating appropriate annotations for input sentences, i.e. by offering a long and detailed list of guidelines that aims to define an annotated entity.

The effectiveness of prompt engineering lies in the fact that modern LLMs exhibit strong sensitivity to the phrasing of the instructions. This is useful as this is exactly what makes modern models flexible, but it is also what makes prompt engineering a challenging task, as designing a complex prompt that addresses several aspects of a given problem is not trivial at all and requires terrific amounts of trial and error. This problem is exacerbated when considering that testing a dataset requires a non-negligible amount of computational resources, making the cycle of design and test of a prompt a very expensive process, besides being very hard to track and debug.

Notable examples of similar guideline-based approaches include structured prompting for complex reasoning [18] and LLM-assisted optical character recognition pipelines [15], where carefully designed prompts were essential to achieve robust performance. These guidelines-based approaches have been inspirational to the design of the proposed pipeline and yield promising results when applied to complex tasks such as

NER.

Finally, a key difference in prompt engineering can be made by the implementation of proper *instruction formats*. Many modern LLMs are fine-tuned using instruction-following datasets that implicitly assume a specific conversational or structural template: when these templates are not respected at inference time, model behavior can degrade substantially. For instance, Llama 3.1 requires a well-defined prompt format [14] to properly distinguish between system instructions, user input, and model responses. The evaluation phase of this project showed that deviating from the expected format often leads to undesirable artifacts, such as repetitive token generation, failure to follow instructions, or uncontrolled continuation beyond the intended output length.

3.4.1 In-Context Learning

Due to the way Large Language Models are trained, they do not acquire task-solving abilities in the classical sense. In standard learning theory, a model is said to learn a task if, after being exposed to a sufficient number of labeled examples, it achieves a bounded probability of error on previously unseen inputs (as seen in Valiant, 1984). In such a setting, learning implies a persistent update of model parameters that improves performance on a well-defined task distribution.

LLMs, however, operate under an entirely different paradigm: once training is complete, their parameters remain fixed at inference time, and instead of adapting through weight updates, these models exploit a phenomenon known as In-Context Learning (ICL). In ICL, the model is able to perform a task specified in natural language by conditioning it on the input context, which may include instructions, examples, or demonstrations of the desired behavior. This is exactly what makes prompt engineering so effective. The model does not "learn" the task by weight

update; instead, it infers the task structure from the prompt and applies it through next-token prediction [31, 7].

This behavior is most commonly observed in few-shot prompting, where a small number of input-output examples are provided within the prompt itself [30]. The model then generalizes the underlying pattern and produces correct outputs for new inputs without any additional training. From a probabilistic standpoint, the model is simply continuing the text sequence in a way that is most likely to occur given this learned distribution, with the final result that the output string is a satisfactory response to the given request.

3.5 Retrieval Augmented Generation

As discussed in the previous sections, Large Language Models are models that are optimized for next-token prediction and are able to provide a number of immense benefits to any natural language task. However, it does not guarantee factual correctness: as mentioned throughout this document, a well known limitation of LLMs is their tendency to produce *hallucinations*, output that is syntactically coherent and contextually plausible but factually incorrect or unsupported by evidence [32]. This issue significantly hurts trust in LLM-generated content, especially in domains where stakes are high such as law, medicine, and research.

Retrieval Augmented Generation (RAG) is a framework designed to mitigate this limitation by combining Large Language Models with classical Information Retrieval techniques. Instead of relying solely on the model's internal parameters as a source of knowledge, RAG systems retrieve relevant information from an external knowledge source and inject this information directly into the model's context. This source of knowledge typically consists of a vector database that has been previously built

specifically for the task at hand. The model then uses both its generation and the retrieved evidence to output an answer to the user query.

In practice, RAG pipelines consist of two main components: a *retriever*, which encodes queries and documents into vector representations and identifies the most relevant passages using similarity search, and a *generator*, which uses the retrieved passages as additional context to produce the final output.

The retriever is the component responsible for accessing external knowledge: it encodes both the user query and the documents using a dense vector representation, typically using a transformer-based embedding model; once encoding is finished, every document is stored in a vector database containing every document, and the codified user query is looked up against this database to find the most relevant documents. This can be achieved through several means, such as BM25 or semantic similarity. BM25 is a clear improvement over semantic similarity for NER specifically. Good retriever performance is necessary to ensure that any RAG pipeline performs properly, and proper algorithm and hyperparameters are extremely important to ensure that the pipeline adapts well to the domain of choice.

The next major component is the generator, represented by a Large Language Model that receives the original query augmented with the retrieved documents as part of its input context. Its objective is to generate a coherent and contextually appropriate response by leveraging both its internal knowledge and the externally retrieved evidence. By grounding the generation process in documents that contain factual and relevant information, the generator can reduce hallucinations, improve factual accuracy, and adapt responses to domain-specific information. In many implementations (such as Haystack, the one used within this project) the generator is also responsible for structuring the output, for example by enforcing formatting constraints or extracting specific pieces of information from the retrieved documents.

This approach offers several advantages, including the reduced hallucinations and improved factual accuracy mentioned above. Additionally, it enables access to up-to-date information and increases transparency by allowing outputs to be traced back to source documents, typically attached to the response of the model. Most importantly: it allows additional information to be injected into the model without having to perform long, expensive finetuning steps.

Retrieval Augmented Generation has become a foundational technique for building reliable LLM-based systems, particularly in applications involving document analysis and especially knowledge extraction. Since there is little literature on using Retrieval Augmented Generation in Named Entity Recognition, this work aims at contributing to that niche.

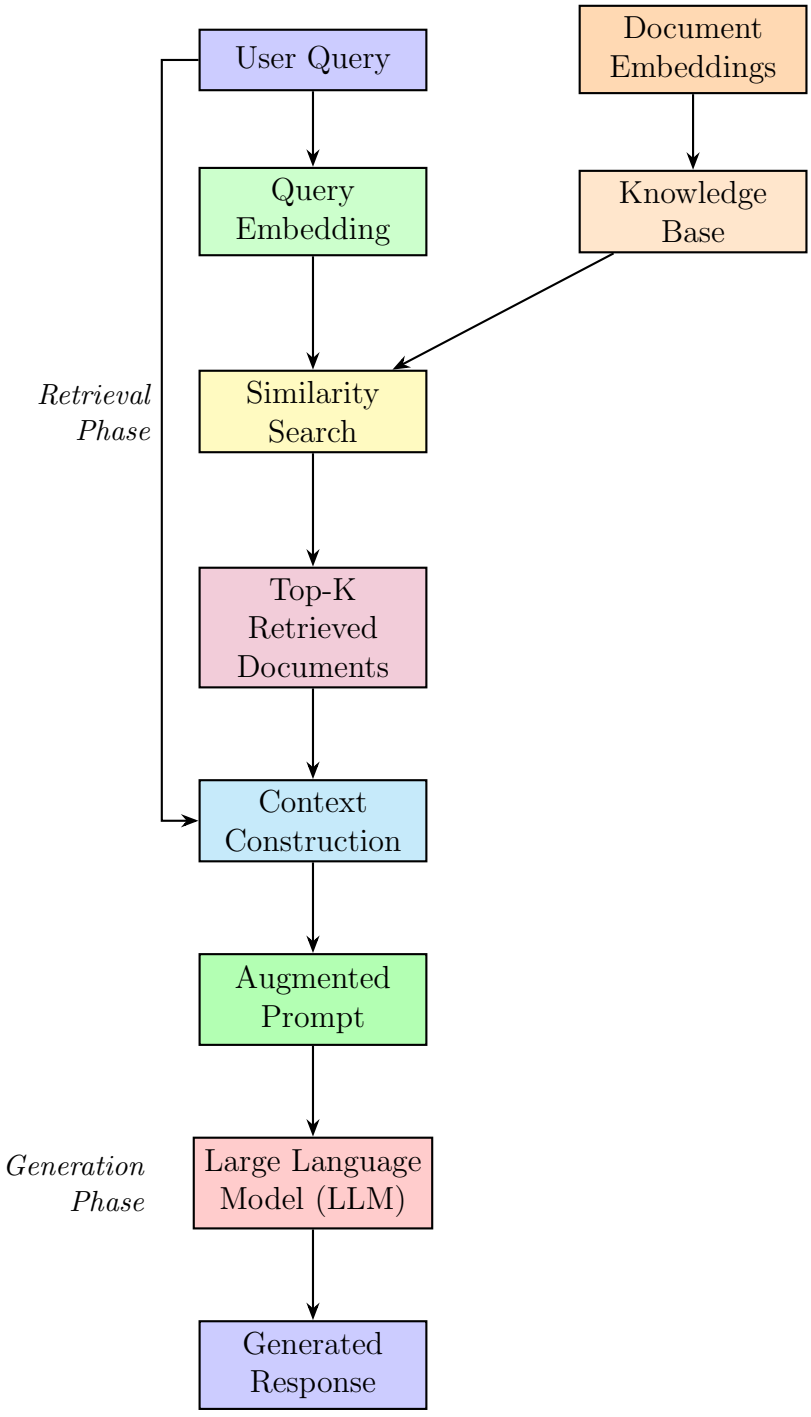


Figure 7: Architecture of a Retrieval Augmented Generation system

4 Approaches for Named Entity Recognition

This chapter illustrates the possible approaches for Named Entity Recognition, focusing on the Transformer-based technologies that have been examined so far. As often iterated through this work, there are fundamental differences between models such as BERT and models such as GPT. These differences can be roughly summarized as differences in training objective: while Autoencoding Transformers are trained by minimizing a loss function on Next Sentence Prediction and Masked Language Modeling, Autoregressive Transformers are trained by minimizing a loss function on next token prediction in a given sentence.

This distinction has several non-trivial implications, namely the fact that these tools must be used in completely different ways. For instance, while a model such as BERT natively excels at tasks such as Text Summarization or Information Extraction, models such as GPT may require additional steps from a prompt engineering perspective purely because the model is not trained for this purpose. In some cases, such as Named Entity Recognition, BERT may easily fit the task more naturally, but with a prompt engineering effort GPT may be able to perform similarly and even outperform BERT in some instances, due to its unique features such as its world knowledge.

The remainder of this thesis attempts to quantify these differences with the intent of bringing a new perspective on the way in which LLMs can be employed to perform Named Entity Recognition.

4.1 Autoencoding Transformer approach

Approaching Named Entity Recognition with Autoencoding Transformers such as BERT involves representing the problem as a token classification task. In this formulation, each token in the input sequence is assigned a label from a predefined set, allowing the model to learn contextualized representations that capture both local and global dependencies. During fine-tuning, the pretrained BERT model is augmented with a token-level classification head, typically implemented as a linear layer followed by a softmax function.

NER datasets are commonly span-based, but they can be represented in different machine-readable formats. For example, given the sentence "Mario lives in Milan," a BIO-encoded dataset may be expressed in various ways:

- **Indexed and labeled spans of tokens**, represented as explicit annotations with character offsets:

  [["Mario", "PER", 0, 5], ["Milan", "LOC", 15, 20]]

- **Sequences of labels**, where each token is assigned a corresponding tag:

  ["PER", "O", "O", "LOC"]

- **CoNLL-style**, multiple columns per token representing different features of the word, such as the token itself, its POS tag, and its NER label.

```
Mario    B-PER
lives    O
in       O
Milan      B-LOC
.        O
```

Both representations are widely used in classical benchmark datasets such as CoNLL2003 [19] and OntoNotes 5.0 [3]. During fine-tuning, these datasets allow the model to learn a probability distribution over token-label associations, leveraging BERT's contextual embeddings to resolve ambiguities and capture long-range dependencies. Despite their effectiveness, BERT-based approaches require substantial labeled data and are typically constrained to fixed label sets defined at training time, making for systems that are robust but not flexible. Additionally, finding an appropriate and sufficient source of data may not be an easy task.

4.2 Autoregressive Transformer approach

NER approaches based on Autoregressive Transformers differ fundamentally from traditional BERT-based methods. In this paradigm, NER is formulated as a generative problem where the model is instructed to output annotations of the input text or a structured representation of extracted entities. Achieving reliable results requires complex prompt engineering to ensure that outputs are both accurate and parsable by the machine. Additionally, hyperparameters such as temperature, top-k sampling, and maximum token length must be carefully controlled in order to enforce deterministic and consistent behavior.

Autoregressive Transformer NER approaches trade explicit supervision for flexibility and adaptability. While they often require careful prompt design and exhibit sensitivity to generation artifacts, they offer a powerful alternative in low-resource set-

tings and applications where rapid prototyping and schema flexibility are paramount, providing a solid set of benefits that create a niche that Autoencoding Transformer methods cannot satisfy.

Unlike BERT-based methods, LLM-driven NER does not rely on traditional token-labeled datasets for fine-tuning. Instead, learning occurs implicitly through either the encoded world knowledge of the LLM or through examples embedded directly in the prompt. As a result, datasets must be reformatted into natural language demonstrations that pair raw sentences with their annotated counterparts. As described in [28], a strong application of LLMs for NER uses few-shot examples consisting of parallel input–output pairs, where entity mentions are marked with special delimiters, enabling automatic parsing of model outputs.

For example, as seen in [28]:

...
Input: Only France and Britain backed Fischler's proposal.
Output: Only @@France## and @@Britain## backed Fischler's proposal.
...

Figure 8: Example of parallel input/output

Presenting this kind of template to the model additionally causes the model to also report output in the same way, both in terms of delimiters and in terms of output layout, basically performing the so called In-Context Learning discussed in section 3.4.1.

This simple approach represents a starting point for NER through Autoregressive Transformers. As seen in section 5.6, this method acts as a solid base for further development.

Part II

The Proposed Approach

This chapter deals with the design, analysis and implementation of a system that proposes solutions to Named Entity Recognition through the use of Large Language Models. As previously mentioned in the appropriate sections, this discussion involves the combination of a plethora of tools such as Large Language Models, Quantization, In-Context Learning, Retrieval Augmented Generation and Prompt Engineering. However, there are some components that deserve the most focus, as these represent the true core of the proposed solution.

The following sections describes in detail the proposed pipeline, able to perform Named Entity Recognition on previously unseen data inputs with little to no prior domain knowledge and especially with no finetuning at all. Additionally, the analysis deals with the analysis of the way in which presence of data actually improves results, offering a way to tangibly measure the benefits contributed by this approach.

5 Methodology

This section aims to provide the methodological evolution of the system developed during this thesis work, starting from the original baseline and moving toward the proposed solution. The objective is to highlight the design, architecture and techniques introduced in this project.

First, the baseline system proposed by Zappata is analyzed, providing an overview of both the juridical dataset and the NER system. Then, the proposed approach is introduced and described in detail, including the novel way in which RAG is employed to achieve NER performances that include benefits that are typically absent from classical BERT pipelines.

In the end, the discussion focuses on benefits, disadvantages and limitations of the proposed method, contrasting it with the baseline architecture in order to assess the actual improvements and unique benefits that were uncovered in the making of this work.

5.1 Problem Statement

As noted throughout this document, Named Entity Recognition is a complex problem that needs accurate and thoughtful design in order to be properly solved, and even more so when the available data is low or the domain is niche. Systems such as SVMs, Deep Neural Networks and especially Autoencoding Transformers (such as BERT) have been shown in previous sections to represent the backbone of the modern State of the Art for NER. However, strictly supervised systems such as these require a huge amount of data in order to be properly trained, which may not be feasible in low-resources scenarios, where data is limited due to the nature of the domain and

obtaining more data may be costly or even impossible. In such cases, these models naturally tend to become overfit and struggle to generalize to unseen data.

Today, Large Language Models are considered to be a good solution to several Natural Language Processing tasks due to their innate generation capabilities. However, LLMs tend to fall short in tasks where generation is not centric to the problem, as is the case for NER, a token classification task. Despite these deficiencies, LLMs present a collection of interesting capabilities for NER, such as vast prior world knowledge, the ability to perform the task without performing an explicit finetuning step and the possibility to control the model through prompt engineering without having to retrain the model in any capacity: all of these points provide benefits that cannot be obtained through traditional methods.

The next sections deal with the design, analysis and evaluation of pipelines that use LLMs to perform NER with the intent of highlighting the unique benefits that emerge when an LLM is used to solve NER. The proposed approach attempts to circumvent the problems posed by Autoencoding Models through the adoption of an entirely different paradigm, involving a set of guidelines and guardrails that enable proper extraction of information while reducing generation artifacts.

5.2 Datasets

The following sections explore the datasets that have been used to test this pipeline.

5.2.1 Kessler Italian Named-entities Dataset (KIND)

The Kessler Italian Named-entities Dataset (KIND) dataset is a collection of four multi-domain Named Entity Recognition datasets proposed by the Digital Humanities Unit at Fondazione Bruno Kessler [4].

Each of these four datasets present NER-annotated sentences in the Italian language, using the CoNLL style that was previously mentioned. It contains more than one million tokens and three classes: person, organization and location.

The datasets are:

- **Wikinews**, presenting Italian articles from the last 20 years;
- **Literature**, presenting sentences from public domain Italian books;
- **Aldo Moro's works**, presenting documents written by Aldo Moro and collected by the University of Bologna;
- **Alcide De Gasperi's writings**, presenting a collection of public documents realized by Alcide De Gasperi between 1901 and 1954.

Table 1: Overview of the KIND dataset

Dataset	**#Entities**	**#PER**	**#LOC**	**#ORG**
De Gasperi's writings	5631	1382	1320	2929
Italian Literature	5738	4076	1196	466
Aldo Moro's works	10348	1741	2831	5776
WikiNews	28724	10731	8575	9418

5.2.2 Juridical Sentences

In addition to the KIND dataset, this dataset, annotated by Zappata in the context of her own thesis work, has been used as a test for the pipeline. It comprises of several Italian legal documents, containing juridical sentences and rulings for a total of 379539 data samples.

The raw dataset comprises of several documents in .docx and .pdf formats. As per Zappata's thesis, each document has been splitted sentence-wise, totalling 379539

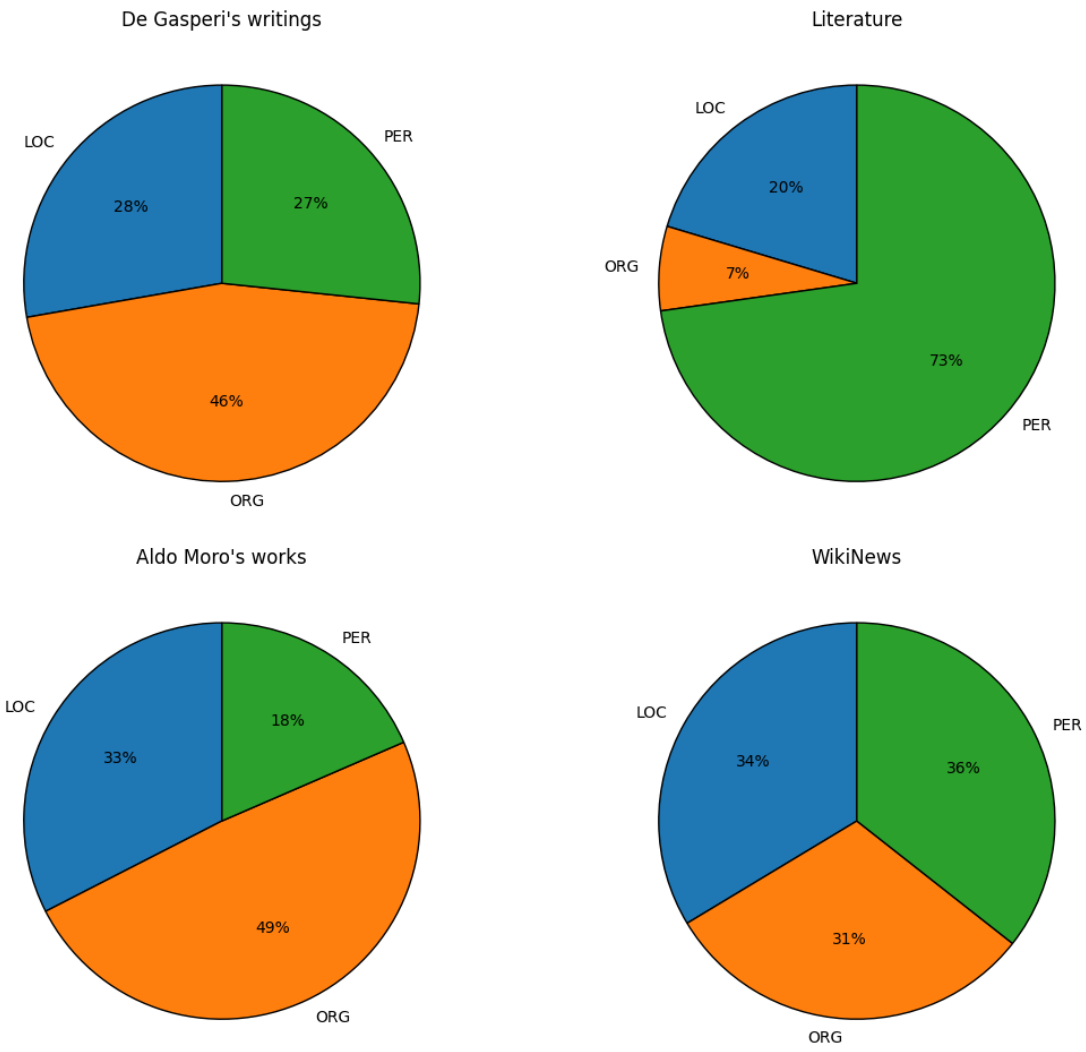


Figure 9: NER Label Distribution for KIND

sentences: 21% coming from .docx files, 79% coming from the .pdf files. This difference is relevant as performing OCR on the PDF documents results in several scanning issues that require proper handling to become usable in the context of this work.

Out of all the sentences, 1028 have been annotated by Zappata using BIO annotations and using four labels: PER, ORG, LOC and MISC. As per figure 10, PER labels comprise the vast majority of annotated labels in the dataset, with LOC and ORG presenting similar numbers and with MISC labels representing just around 1% of the dataset. This number of PER entities is expected given the nature of the dataset, since juridical sentences tend to present long lists of defendants, judges, attorneys and parties involved in said sentences, in general. Since this imbalance is significant, it may be appropriate to account for it, for example by using stratification when training a model.

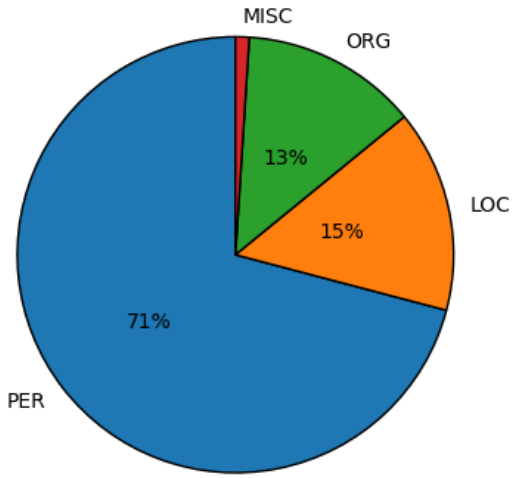


Figure 10: Label Distribution for the Juridical Sentences Dataset from Zappata

5.3 Baseline

The baseline for this project was originally developed by Zappata in the context of her own master's thesis, and serves as the reference system against which the proposed approach is compared. The baseline is centered around several BERT-like autoencoding transformer model, finetuned for Named Entity Recognition on the above-mentioned dataset of juridical sentences.

The pipeline architecture consists of a standard supervised NER system. The task uses BERT [9] to solve Named Entity Recognition by solving a token classification problem, assigning a label from the predefined set of PER, LOC, ORG, MISC, using standard BIO: all tokens are prefixed with either B- or I- unless it is not recognized as an entity, in which case O is assigned. Since this system relies on BERT, it exploits the ability of BERT to account for context: for this very reason, this baseline is able to assign labels that are actually meaningful to the current phrase.

The original implementation was developed using spaCy as its primary framework, used for both model training and inference. As per the original thesis, spaCy was used for the initialization, training and evaluation of the BERT models.

The results published by Zappata have been successfully reproduced in the initial phases of this work. Differently from the original implementation, the transformers library developed at HuggingFace has been used instead. This re-implementation preserved the original models, training objectives, hyperparameters and dataset, while relying on the new standard, modern and flexible foundation that could be later used to test new models in the future. The results of the re-implementation can be seen in table 5.6.1.

Additionally, since the results produced by Zappata only include the juridical sentences dataset, the re-implementation of the baseline is additionally ran on the

KIND dataset, whose results are shown in table 5.6.1. The model used for this is the best model out of 5.6.1.

5.4 Proposed Method

The proposed approach is motivated by the assumption that Large Language Models can perform Named Entity Recognition when provided with carefully selected examples that are similar to the target input. Differently from BERT-like models, instead of relying on finetuning, the method retrieves documents that contain annotated entities, and embeds them into a properly structured prompt.

This approach is based on the execution of several steps, requiring the joint usage of (1) a data format that is suitable for LLMs, (2) a retrieval system, (3) a well-engineered prompt, (4) an LLM and (5) a set of relevant documents to use in few-shot learning. Figure 5.4 displays the full architecture.

Combined together, these components form a NER pipeline that integrates principles of Retrieval Augmented Generation, specifically through the implementation of the retrieval module. This component allows the system to guide the model into returning outputs that are appropriate to the input query, conditioning the generation into considering tokens related to the retrieved samples.

5.4.1 Data Format

Both the KIND dataset and the dataset by Zappata present data in a classical BIO format, where each data sample is tokenized and labeled with B- (Beginning), I- (Inside) or O (Outside) tags, concatenated to one of the labels for the task, such as PER, LOC or ORG.

However, BIO is not suitable for text extraction tasks using LLMs, as these models are designed for natural text generation, and querying the LLM to return a list of labels has proven to be an unviable route, yielding results that are far from ideal due to its subjectivity to hallucinations and generation artifacts in general. As noted by

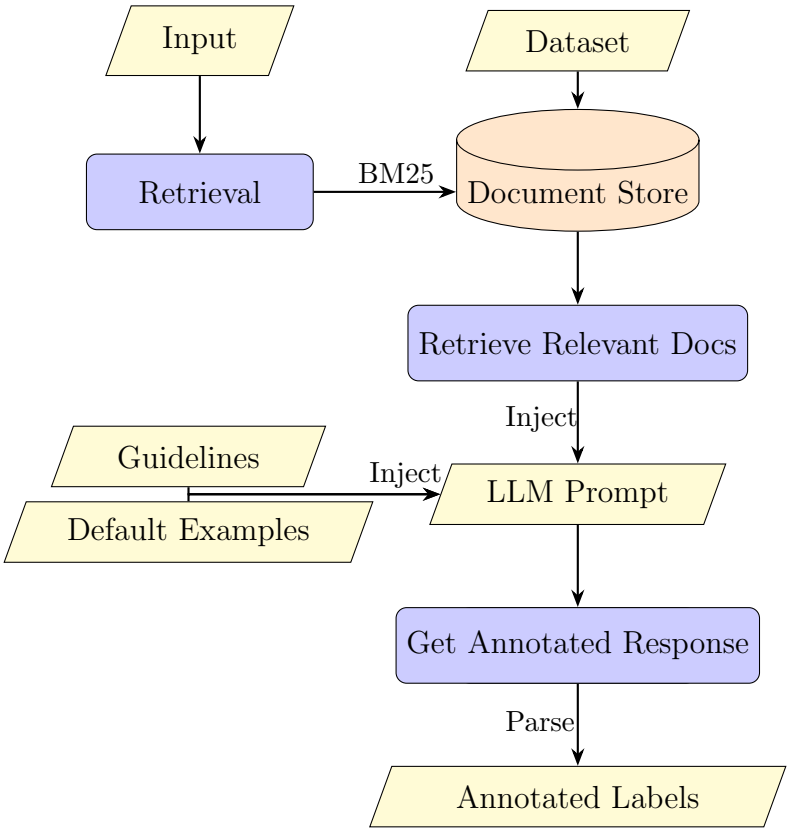


Figure 11: NER System Architecture with RAG Retrieval and LLM

existing research such as [28] and confirmed during experimentation, LLMs perform better when queried to return the input text with annotations, compared to when it is queried to produce a list of tokens. This behavior especially makes sense when considering that LLMs are known to have improved performance when iterating on a given input through intermediate steps [29], which in this work are provided by the combination of guidelines, several relevant examples and a template for model input/output.

For all of the above reasons, an alternative representation for data samples has been designed within this thesis with the intent of producing outputs that are more appropriate and "natural" for LLM generation, are easy to parse and are conceptually easy to grasp.

The parallel format dataset and the annotations used in the context of this work are defined as follows, in table 2.

Table 2: Annotation format used for entity labeling

Entity Type	**Tag**	**Delimiter Format**	**Example**
Person	PER	((entity))	((Pier))
Location	LOC	[[entity]]	[[Perugia]]
Organization	ORG	{{entity}}	{{Eni}}
Miscellaneous	MISC	<<entity >>	<<25-12-2025 >>

For example, a sentence annotated using this format would be represented as

"((Giorgio)) lives in [[Palermo]] and works at {{UNICEF}}."

This format immediately allows for an automated way to extract entities, since extracting these is only a matter of applying Regular Expression defined as in table

3 to any given string. Most importantly, this format is easily convertible across other common representations of NER datasets, including the indexed, sequencial and CoNLL styles representations described in section 4.1. This makes the adoption of this format trivial, as it is just a matter of rewriting the inputs as a sequence of tokens in natural language, except the annotated entities are now surrounded by the double brackets described in table 2.

In the proposed implementation, the re Python library was used in order to match these patterns and extract entities from the strings.

Table 3: Regex patterns and their corresponding entity tags

Regex Expression	**Tag**
`\(\((.*?)\)\)`	PER
`\[\[(.*?)\]\]`	LOC
`\{\{(.*?)\}\}`	ORG
`\<\<(.*?)\>\>`	MISC

As a consequence of the above points, the original dataset has been converted in a parallel format so that the model is able to see both the input and the output for any given data sample, and use this representation for few-shot learning and evaluation. Additionally, the model follows the template in figure 14 when performing prediction through text generation.

BIO Format

```
{
  "id": "1",
  "text": "Mario Rossi lives in Milan",
  "label": [[0,5,"B-PER"],[6,11,"I-PER"],
  [12,17,"O"],[18,20,"O"],[21,26,"B-LOC"]]
}
```

Parallel Format

```
{
  "id": "1",
  "text": "Mario Rossi lives in Milan",
  "pred": "((Mario Rossi)) lives in [[Milan]]"
}
```

Figure 12: Comparison of data formats

5.4.2 Retrieval System

In this work, the retrieval component of the pipeline was implemented using the Haystack framework, which provides modular support for both classical retrieval algorithms such as BM25 and sentence similarity ones.

During experimentation, it quickly became clear that BM25 vastly outperforms sentence similarity algorithms. Sentence similarity tends to retrieve sentences that are similar in meaning, using sentence embeddings to assess similarity between sequences

of tokens. However, for our use case it is much more effective to retrieve documents that present the same keywords as in the input query, which is effectively where BM25 excels. In fact, the score produced by BM25 tends to increase when, among other criterions, the term is rare within the corpus. This factor causes BM25 to prefer documents that present specific keywords, which is appropriate for the retrieval of documents that present the same entities and are already annotated within the document store.

For example, given an input query such as:

((Sofia)) likes to eat caramelized onions at {{Bianchi's}}

The retrieval system works as follows:

1. **Query Encoding**: the input sequence is transformed into a numerical representation suitable for retrieval. For example, this could be either a sparse representation based on TF-IDF in the case of BM25, or a representation such as embeddings in the context of sentence similarity.

2. **Similarity Computation**: the encoded query is compared against every document contained within the document store. Tokens such as "Sofia" and "Bianchi's", are receive higher IDF, contributing much more to the score than common tokens such as prepositions, articles and other grammatical constructs, which tend to receive lower IDF values.

3. **Document Ranking**: every retrieved document is ranked on the basis of a score, related to the chosen retrieval ranking function. The *top_k* documents are returned. Since these documents are ranked, it is likely that the top ones contain annotated entities that are useful to our input.

Since every document in the document store is already annotated, the retrieved documents is plausibly composed by documents that are useful to annotate the new input query.

This component of the pipeline is vital and is greatly effective in improving system performance. This is accurately tracked and measured in the Results section. Also, differently from the baseline, this aspect of the pipeline makes the system uniquely interpretable, since there is now a clear, human-readable flow of information that can be used to troubleshoot issues in system performance.

5.4.3 Guidelines Prompt

An extensive prompt engineering phase was undertaken with the intent of constructing a robust, complete prompt that guides the model into accurately identifying and annotating entities within any arbitrary sentence. Rather than providing a minimal task description for the NER task, the prompt has been deliberately designed in the form of a lengthy set of guidelines.

Since Named Entity Recognition is such a complex task, where there are many rules, definitions, exceptions and ambiguities to account for, deciding to opt for a bigger prompt was an easy choice. Although it may intuitively be easy to identify an entity within a string, the same cannot be said when trying to guide a model into doing so. These intuitions justify the choice of a longer prompt.

The guidelines are defined in a very comprehensive way. At the top of the prompt, an introductory statement is used to set the intent for the model, defining it as a NER annotator that has to solve NER on the input. This portion of the prompt is also used to define tags and delimiters. Then, a comprehensive list of guidelines is described, along with several meaningful examples for each guideline. A portion of the guidelines is shown in figure 13.

The guidelines on which the system has been tested is in Italian and possesses a huge count of 807 words for a total of 5693 characters, also counting the characters introduced by the prompt formatting tags.

You are a NER annotator. You must perform Named Entity Recognition (NER) on the input documents. Entities must be surrounded by the following indicators:
- Person (PER): ((...))
- Location (LOC): [[...]]
- Organization (ORG): {{ ... }}
- Miscellaneous (MISC): <<... >>
1. An annotation is a substring that can be associated with a specific label, such as PER, LOC, ORG, or MISC. Not all input sentences contain annotations: don't annotate unless you detect an entity.
2. Words must be or must contain proper nouns:
- "Mario Rossi" consists of two consecutive proper nouns that represent a single entity.
- Names may include words that are not proper nouns, such as "Mario da Catania".
- "Belmonte Mezzagno" is the name of a place but is composed of two consecutive common words: "Belmonte" and "Mezzagno".
- The entity must be constant over time: "the president of Enel" is not a person's name, since it changes over time, while "Rossi" is a name because the entity it refers to does not change.
3. You will annotate entities using the following labels: Person (PER), Organization (ORG), Location (LOC), and Miscellaneous (MISC).
IMPORTANT: Return text in output exactly as it is described in the input. Return proper names as they appear in the input.
IMPORTANT: Immediately interrupt output after the annotated text. Do not provide explanations or comments to the output.

Figure 13: Reduced version of the guidelines used in the pipeline.

These guidelines serve a dual purpose: first, they establish a definition of what constitutes an entity, defining in a fine grained way what an entity is. Second, they impose structural and formatting constraints on the model's output, which, combined with the data format design decisions taken in section 5.4.1, guide the model into

generating sequences of tokens that correctly solve the Named Entity Recognition task. After every major guideline, a set of examples is provided in order to give further context for entities.

Following the guidelines is a list of "default" input/output examples, implementing aspects of few-shot learning by being present in every query sent to the model. These are meaningful examples that aim to capture aspects of different labels, besides providing a format for input and output.

Input: Mario lives in Milan and works for Eni.
Output: ((Mario)) lives in [[Milan]] and works for {{Eni}}.

Figure 14: Example of input/output.

5.4.4 Large Language Model

Large Language Models represent the final key component of this pipeline, acting as a juncture point between every other piece. The choice of a suitable LLM and proper model provider influences choices in the rest of the pipeline. In the context of this project, factors such as determinism, context length, quantization support and general flexibility are highly needed knobs

The transformers library realized by HuggingFace was previously used as the model provider in several phases of the project, such as within the implementation of the baseline and within the LLM component of the pipeline itself. However, factors such as model context length and quantization support quickly became important bottlenecks that needed addressing before being able to move further in the project. For this reason, despite being implemented using HuggingFace, this pipeline has later been re-implemented using Ollama instead, which was then used as the framework of

choice even on pipeline evaluation.

Various models have been examined for this work, such as Gemma 3 [24], DeepSeek-R1 [11] and Llama [25]. Since the initial phases of this project, Llama3.1 is the one that has consistently outperformed every other model, and has since been picked as the model of choice on which all later testing has been performed.

Similarly to other models, Llama 3.1 requires a specific prompt format in order to perform as expected [14]. For this reason, several efforts have been made in constructing a query that respects the format specified by Meta in the model card. During testing, this was immediately evident as the model demonstrated erratic behavior such as generation artifacts, which immediately dissolved once the prompt had been adjusted with the proper template. This single change instantly eliminated generation artifacts such as loops of tokens or generation halting before expected, which would yield unusable output.

Most of the model setup was limited to temperature, context window length and quantization. Since this problem inherently requires structured output, the model temperature was set to 0 in order to ensure determinism. Context window length was set to 64,000 for 8B model, but was set to 10,000 for the 70B model to prevent the system's memory from becoming saturated, crashing the entire machine.

Execution-wise, the only task for this model is to receive the input from the previous parts of the pipeline and process it. The input is constructed as the following string, which is the concatenation of different outputs of the pipeline:

$$\text{Query} = \text{Guidelines} + \text{Default Examples} + \text{Retrieved Docs} + \text{Input Sentence} \quad (5)$$

This concatenation directly matches the data flow in figure 11, and the model directly addresses the Input Sentence, producing output that uses the exact same

template shown in figure 14. This ensures that the output remains easy to parse. Once the model has produced the output, the annotated result can be finally used for downstream tasks.

5.5 Experimental Setup

This section describes the environment in which all the experiments have been performed, including the validation strategy and the evaluation method adopted to assess the proposed pipeline.

5.5.1 Technical Environment

All experiments were conducted on a workstation provided by the Dipartimento di Matematica e Informatica at the Università degli Studi di Palermo (UNIPA), accessed through SSH. The machine presents an Intel 12th Generation i7-12700F CPU, 48 GB of RAM and an NVIDIA RTX 6000 GPU. The high availability of VRAM and RAM is essential for running large language models with extended context windows, particularly for the 70B parameters variant used in this work.

The whole pipeline is implemented as a Python script and associated modules. External libraries such as scikit-learn were also essential. Package and project management was realized using uv. As mentioned in earlier sections, Ollama was used as the model provider. The juridical sentences dataset has been processed using the llama3.1:8b and llama3.1:70b images, while the KIND dataset was processed using just the 8B variant. Both of these model images were downloaded within the Ollama environment and accessed through the Haystack Ollama integration, which communicates with Ollama through an endpoint served by the Ollama executable itself. Both the models were quantized using 4-bit quantization.

The Retrieval component, including the support for the document store and the BM25 ranking function, has been implemented using the Haystack framework, offering support for Retrieval-related tasks such as RAG.

5.5.2 Validation Strategy

To ensure a robust evaluation of the pipeline, a 10-fold cross-validation procedure has been adopted: the dataset was partitioned into 10 disjoint splits of approximately equal size. On each split, train and test indices are used to divide the data into train/test sets: the train set is inserted into a newly initialized document store, while the test set is processed by the entire pipeline, returning annotated entities that then undergo the evaluation procedure in which the predicted annotations are compared to the ground truth labels.

5.5.3 Evaluation Strategy

The pipeline performance was evaluated while accounting for different values of top_k, i.e. different amounts of retrieved documents. This is crucial to properly assess how well this aspect of the system behaves. For each top_k value, varying in $[0, 7]$, 10-fold cross-validation is performed, and performance is evaluated at each fold; after every fold has been processed and evaluated, the average of each fold for Precision, Recall and F1 is used as the score for the current top_k. Standard deviation for the same statistics is also computed. Additionally, label-wise average and standard deviation are computed, enabling a more fine-grained analysis.

The evaluation procedure for a given value of top_k can be summarized as follows:

1. Query the document store with the input sentence.

2. Retrieve the top_k ranked documents.

3. Inject the retrieved documents into the prompt.

4. Generate entity annotations using the language model.

5. Compare the generated annotations against the ground truth.

5.6 Results

This section presents the evaluation of the proposed Retrieval-Augmented Named Entity Recognition Pipeline. There are two main objectives to these experiments: first, to assess the performance of the system in comparison to the baseline model; second, to analyze the impact of retrieval on performance, with particular attention to the number of retrieved documents top_k. Additionally, the impact of the number of parameters is very briefly noted.

Every reported result has been produced using the setup described in section 5.5. Additionally, the performance of every dataset is assessed on multiple levels: it is assessed based on top_k values, but it is also assessed while accounting for different labels.

5.6.1 Baseline Evaluation

The baseline system described in section 5.3 has been re-implemented and evaluated according to the original description. The obtained results for the juridical sentences dataset are shown in figure 5.6.1.

Model	Precision	Recall	F1-Score
bert-base-italian-xxl-cased	**0.915**	**0.920**	**0.917**
hseBert-it-cased	0.898	0.919	0.908
roberta-base	0.874	0.897	0.885
distilbert-base-uncased	0.788	0.824	0.805

Table 4: Baseline NER Evaluation on juridical sentences dataset

The models hseBert-it-cased and bert-base-italian-xxl-cased clearly outperform the other two systems. An easy explanation for this behavior is due to the fact that both of these two models are finetuned on Italian documents. The top 1 model, bert-base-italian-xxl-cased, has been extensively finetuned on Wikipedia articles in Italian and on the OPUS corpora collection. Interestingly, although hseBert-it-cased is directly derived from bert-base-italian-xxl-cased and additionally trained on Italian regulation texts, its performances slightly lag behind.

Additionally, the following table collects results for the top baseline model applied to the KIND dataset.

Dataset	Precision	Recall	F1-Score
De Gasperi's writings	0.842	0.828	0.835
Italian Literature	0.807	0.824	0.815
Aldo Moro's works	0.948	0.966	0.957
WikiNews	0.880	0.902	0.890

Table 5: Baseline NER Evaluation on KIND dataset using bert-base-italian-xxl-cased

It's to be noted that results on some of these datasets, namely De Gasperi's writings and Italian Literature, are underwhelming when compared to the remaining two datasets.

5.6.2 Pipeline Evaluation on KIND dataset

This section presents the results obtained from evaluating the pipeline on each of the KIND datasets, evaluated as described in the previous sections and compared against the baseline results that were just illustrated. As mentioned in the experimental setup section, these datasets have been processed by Llama3.1 8B only.

top_k	**Precision**	**Recall**	**F1 Score**
0	0.541 ± 0.047	0.633 ± 0.045	0.583 ± 0.044
1	0.598 ± 0.059	0.682 ± 0.049	0.637 ± 0.053
2	0.650 ± 0.058	0.731 ± 0.051	0.688 ± 0.055
3	0.667 ± 0.051	0.742 ± 0.041	0.702 ± 0.046
4	0.683 ± 0.053	0.756 ± 0.044	0.717 ± 0.049
5	0.698 ± 0.044	0.772 ± 0.039	0.733 ± 0.041
6	0.712 ± 0.045	0.784 ± 0.047	0.746 ± 0.045
7	$\mathbf{0.721 \pm 0.049}$	$\mathbf{0.789 \pm 0.047}$	$\mathbf{0.753 \pm 0.047}$

Table 6: Dataset evaluation using Llama3.1 8B (KIND - De Gasperi's writings)

top_k	**Precision**	**Recall**	**F1 Score**
0	0.693 ± 0.046	0.788 ± 0.039	0.737 ± 0.044
1	0.775 ± 0.054	0.859 ± 0.044	0.815 ± 0.051
2	0.822 ± 0.055	0.891 ± 0.043	0.855 ± 0.051
3	0.841 ± 0.050	0.899 ± 0.038	0.869 ± 0.047
4	0.853 ± 0.049	0.906 ± 0.039	0.878 ± 0.046
5	0.859 ± 0.050	0.910 ± 0.039	0.884 ± 0.047
6	0.867 ± 0.051	0.912 ± 0.041	0.889 ± 0.048
7	$\mathbf{0.874 \pm 0.054}$	$\mathbf{0.918 \pm 0.043}$	$\mathbf{0.895 \pm 0.050}$

Table 7: Dataset evaluation using Llama3.1 8B (KIND - Italian Literature)

top_k	Precision	Recall	F1 Score
0	0.693 ± 0.050	0.783 ± 0.048	0.735 ± 0.049
1	0.763 ± 0.047	0.848 ± 0.045	0.803 ± 0.046
2	0.807 ± 0.035	0.883 ± 0.031	0.843 ± 0.033
3	0.834 ± 0.033	0.900 ± 0.029	0.866 ± 0.032
4	0.852 ± 0.027	0.916 ± 0.021	0.883 ± 0.025
5	0.861 ± 0.024	0.920 ± 0.017	0.890 ± 0.021
6	0.875 ± 0.023	0.931 ± 0.016	0.902 ± 0.020
7	$\mathbf{0.882 \pm 0.022}$	$\mathbf{0.934 \pm 0.015}$	$\mathbf{0.907 \pm 0.020}$

Table 8: Dataset evaluation using Llama3.1 8B (KIND - Aldo Moro's works)

top_k	Precision	Recall	F1 Score
0	0.607 ± 0.018	0.704 ± 0.022	0.652 ± 0.019
1	0.674 ± 0.015	0.759 ± 0.019	0.714 ± 0.015
2	0.715 ± 0.017	0.787 ± 0.017	0.750 ± 0.017
3	0.737 ± 0.015	0.801 ± 0.018	0.768 ± 0.015
4	0.754 ± 0.016	0.812 ± 0.017	0.782 ± 0.016
5	0.765 ± 0.017	0.821 ± 0.018	0.792 ± 0.018
6	0.775 ± 0.019	0.831 ± 0.019	0.802 ± 0.019
7	$\mathbf{0.781 \pm 0.014}$	$\mathbf{0.836 \pm 0.016}$	$\mathbf{0.807 \pm 0.014}$

Table 9: Dataset evaluation using Llama3.1 8B (KIND - WikiNews)

Additionally, table 10 presents a summary of the best model for each dataset, which always corresponds with the system that retrieved the highest number of documents.

Dataset	Precision	Recall	F1 Score
De Gasperi's Writings	0.721 ± 0.049	0.789 ± 0.047	0.753 ± 0.047
Italian Literature	0.874 ± 0.054	0.918 ± 0.043	0.895 ± 0.050
Aldo Moro's Works	0.882 ± 0.022	0.934 ± 0.015	0.907 ± 0.020
WikiNews	0.781 ± 0.014	0.836 ± 0.016	0.807 ± 0.014

Table 10: Summary of evaluation of KIND on Llama3.1 8B

Finally, table 11 extensively details how performance varies for each label.

Dataset	PER			LOC			ORG		
	P	R	F1	P	R	F1	P	R	F1
De Gasperi's Writings	0.935	0.938	0.936	0.834	0.835	0.834	0.730	0.736	0.733
Italian Literature	0.923	0.932	0.927	0.916	0.905	0.910	0.953	0.953	0.953
Aldo Moro's Works	0.973	0.973	0.973	0.934	0.943	0.938	0.896	0.905	0.900
WikiNews	0.920	0.924	0.922	0.840	0.847	0.843	0.822	0.810	0.816

Table 11: Per-label evaluation of KIND on Llama3.1 8B - best model only (top_k=7)

5.6.3 Pipeline Evaluation on Juridical Sentences dataset

As can be observed in tables 12 and 13, increasing the number of parameters noticeably improves the performance of the system. Specifically, the best model for 70B provides a noticeable improvement of 4.2% over the best 8B performance, which brings the system on par with some of the baseline models for this dataset.

top_k	Precision	Recall	F1 Score
0	0.776 ± 0.073	0.756 ± 0.059	0.765 ± 0.065
1	0.824 ± 0.073	0.792 ± 0.064	0.807 ± 0.068
2	0.819 ± 0.057	0.799 ± 0.049	0.808 ± 0.052
3	0.837 ± 0.033	0.811 ± 0.038	0.823 ± 0.035
4	0.841 ± 0.036	0.814 ± 0.030	0.828 ± 0.033
5	0.839 ± 0.038	0.807 ± 0.032	0.823 ± 0.034
6	0.844 ± 0.045	0.800 ± 0.039	0.821 ± 0.040
7	$\mathbf{0.860 \pm 0.054}$	$\mathbf{0.819 \pm 0.047}$	$\mathbf{0.836 \pm 0.050}$
8	0.854 ± 0.057	0.822 ± 0.051	0.835 ± 0.054

Table 12: Metrics across different values of top_k for Llama3.1 8B

top_k	Precision	Recall	F1 Score
0	0.821 ± 0.017	0.844 ± 0.019	0.832 ± 0.016
1	0.844 ± 0.016	0.865 ± 0.017	0.854 ± 0.014
2	0.857 ± 0.014	0.872 ± 0.020	0.864 ± 0.014
3	0.863 ± 0.017	0.875 ± 0.022	0.869 ± 0.017
4	0.868 ± 0.017	0.878 ± 0.021	0.873 ± 0.017
5	$\mathbf{0.873 \pm 0.020}$	$\mathbf{0.876 \pm 0.027}$	$\mathbf{0.875 \pm 0.022}$
6	0.871 ± 0.014	0.874 ± 0.026	0.872 ± 0.018
7	0.875 ± 0.019	0.869 ± 0.035	0.872 ± 0.026
8	0.873 ± 0.015	0.859 ± 0.027	0.866 ± 0.019

Table 13: Metrics across different values of top_k for Llama3.1 70B

These performances demonstrate that LLMs can compete with systems that are designed around BERT-like models. This is an especially respectable achievement when considering that LLMs are not designed for information extraction: this noticeable result is only a by-product of text generation combined with a set of working prompt engineering practices, and these results additionally provide evidence that the number of parameters of the model can improve overall system performance.

Dataset	PER			LOC			ORG		
	P	R	F1	P	R	F1	P	R	F1
Llama3.1 8B - top_k=7	0.945	0.938	0.941	0.730	0.695	0.712	0.771	0.716	0.742
Llama3.1 70B - top_k=5	0.946	0.940	0.943	0.836	0.848	0.842	0.844	0.820	0.820

Table 14: Per-label evaluation of Juridical Sentences dataset on Llama3.1 8B and 70B - best models only

5.7 Discussion

The data presented in the previous section offers a way to analyze the behavior of the proposed pipeline, the impact of retrieval and the competitiveness of LLM-based approaches against standard methods such as BERT-like baselines. The following sections interpret these results and aims to offer interesting perspectives on what was achieved during this thesis.

5.7.1 Comparison against the Baseline

On the juridical sentences dataset, the strongest LLM configuration, presenting Llama3.1 70B at $top_k = 5$, yields an F1 of 0.875. This approaches the lower end of the BERT-based baseline results, and notably surpasses distilbert-base-uncased (F1 of 0.805) and roberta-base (F1 of 0.885). While it does not yet match the top model bert-base-italian-xxl-cased (F1 of 0.917), the gap is considerably closer than one might expect given the fundamental architectural difference between the two approaches, making the competitive performance especially noteworthy.

On the KIND datasets, the situation is more nuanced: the 8B model surpasses the BERT baseline on Italian Literature (F1 of 0.895 vs. 0.815) and performs comparably on Aldo Moro's works (0.907 vs. 0.957), while remaining considerably below the baseline on WikiNews. The most pronounced gap is on De Gasperi's writings, where both the baseline and the proposed system yield relatively low results, suggesting that this dataset presents noticeable difficulties.

The label-wise analysis further provides insight into the way in which the system performs. Across all instances, PER entities are the ones that are identified with the highest precision and recall. LOC and ORG entities prove more challenging, presenting varying degrees of performance that are mostly addressed by the use of

the 70B model. For instance, LOC scales from 0.712 at 8B to 0.842 at 70B.

Overall, the quantitative evaluation tends to reveal several kinds of results: in some areas, such as the Italian Literature dataset, the proposed approach clearly outperforms the baseline; in others, the method is remains comparable with the baseline. This said, it is important to additionally consider the qualitative benefits brought by adopting this method for Named Entity Recognition, which brings unique qualitative benefits that traditional approaches do not possess:

- **Robustness in low-resource scenarios**: this method does not rely exclusively on large quantities of annotated data. As shown in the analysis that present no retrieved documents (top_k=0), the performance still remain modest, indicating that the system is still well-responsive in such circumstances. This characteristic could be particularly advantageous in low-resources or early-stage settings, where annotated corpora is limited or in construction. This could accelerate the otherwise tedious manual annotation process, significantly benefiting to NER projects.

- **Extreme flexibility without explicit fine-tuning**: differently from conventional transformer-based token classifiers, which require retraining or fine-tuning, the proposed method can be adapted through modifications to the prompt structure or retrieval corpus. This significantly lowers the cost and time associated with transfer learning, offering greater flexibility in practical applications.

- **Detection of previously unseen or rare entities**: as a result of the innate world knowledge of LLMs, the proposed method is able to recognize and automatically annotate entities that have not previously been seen. Although hard to quantify, it was found during experimentation that this system could

identify False Positives that were actually perfectly valid annotations, that were however unseen by the original manual annotators. As a consequence, such a pipeline is able to occasionally provide valid data samples that would otherwise remain unannotated.

5.7.2 Impact of Retrieval on Performance

The most consistent finding across all datasets is the improvement in performance as *top_k* increases. This pattern is verified across all four KIND subsets and is largely replicated on the juridical sentences dataset, suggesting that retrieved examples provide useful context to the language model, rather than introducing noise or confusion. The improvement from $top_k = 0$ to the best-performing retrieval configuration is very noticeable in every test. For instance, on De Gasperi's writings the F1 score rises from 0.583 to 0.753, while on Aldo Moro's works the gain spans from 0.735 to 0.907. This confirms that examples retrieved using BM25 act as an effective form of few-shot learning, guiding the model into better performing named entity recognition for the specific domain and annotation schema of each dataset.

It is worth noting that improvements tend to decelerate as *top_k* increases, rather than increasing linearly. This indicates that the most informative examples are likely retrieved early in the ranking, and further documents introduce a comparatively smaller gain in context. Despite this fact, it might be worth it to explore and study these effects for even bigger amounts of retrieved documents.

Even though this pattern holds true for every other configuration, it is interesting to note that the 70B model on the juridical sentences dataset peaks at $top_k = 5$, potentially indicating that the useful informational content was already saturated at that point.

5.7.3 Effects of Model Scale

The comparison between Llama3.1 8B and 70B on the juridical sentences dataset provides a clear indicator that model scale is a meaningful factor in the performance of this pipeline. First, the 70B model exhibits a substantially higher overall F1 (0.875 vs 0.836), and second, it presents a much smaller variance across folds, suggesting that the quality of the predictions is overall more stable compared to the 8B model. Additionally, the per-label analysis in table 14 shows that the 70B model has overall better performance on the ORG and LOC predictions as well, confirming even more that the increase in scale is meaningful in the improved identification of entities.

5.7.4 Limitations and Future Directions

Several limitations of the proposed approach deserve acknowledgment. First of all, this method was tested on a limited amount of models: although this was tested using 8B and 70B LLMs, an interesting direction would be to further test this on different amounts of parameters, measuring system performance as the count varies in a wider range. Most notably, this approach was not tested on a fine-tuned model, which could result in further improvements to the annotation capabilities of this system.

Additionally, it would be interesting to analyze the way in which these datasets vary as the *top_k* value varies. Unfortunately, this analysis is limited by the available VRAM, as the model context window length directly impacts how much dedicated GPU memory is needed for execution. Then, evaluation was conducted exclusively on Italian datasets using a prompt in the Italian language, and remains to be seen how well this applies to other languages that present different structure and conventions.

Conclusions

This thesis has investigated the ways in which Large Language Models can be employed to perform Information Extraction tasks such as Named Entity Recognition, leveraging Retrieval Augmented Generation techniques and thus proposing an alternative to the traditional approaches based on autoencoding transformers.

The central hypothesis around which this thesis revolves is that a Large Language Model, equipped with an extensive, well-designed prompt can be mixed with interesting retrieval heuristics to perform few-shot learning, achieving Named Entity Recognition in a way that performs comparably to traditional BERT-like methods. To test this, a complete pipeline was designed and implemented, integrating (1) a parallel annotation format designed for text generation, (2) a BM25-based retrieval mechanism, (3) a carefully engineered set of guidelines and few-shot examples and (4) a Large Language Model. This was then thoroughly tested using 10-fold cross-validation on several Italian datasets, that included domains spanning across juridical and literary, and were then compared to a strong BERT baseline.

The results demonstrates that the proposed approach is competitive with traditional architectures, surpassing the baseline on some datasets and approaching it on others. Most importantly to note is the fact that increasing the number of retrieved documents directly increases F1 across every dataset, confirming that retrieval acts as an effective mechanism for injecting context into a given language model. It was additionally found that increasing the size of the model yields very measurable improvements in overall performance and stability.

In conclusion, beyond quantitative results, this work reveals important qualitative advantages that are brought by the implementation of this method, such as robustness in low-resource scenarios, modest performance even without retrieved examples, and

ease of adaptation to new domains despite there not being explicit fine-tuning. This work provides solid evidence that Retrieval Augmented Generation, when combined with Large Language Models, presents a set of competitive advantages for Named Entity Recognition over the classical BERT-based systems, offering a flexible and interpretable way to approach NER even in situations where there is little to no available data.

References

[1] URL: https://cs.stanford.edu/people/eroberts/courses/soco/projects/2004-05/nlp/overview_history.html.

[2] URL: https://www.cnbc.com/2025/06/09/openai-hits-10-billion-in-annualized-revenue-fueled-by-chatgpt-growth.html.

[3] URL: https://catalog.ldc.upenn.edu/LDC2013T19.

[4] URL: https://github.com/dhfbk.

[5] Joshua Ainslie et al. "Gqa: Training generalized multi-query transformer models from multi-head checkpoints". In: Proceedings of the 2023 Conference on Empirical Methods in N 2023, pp. 4895–4901.

[6] Iz Beltagy, Matthew E Peters, and Arman Cohan. "Longformer: The long-document transformer". In: arXiv preprint arXiv:2004.05150 (2020).

[7] Tom Brown et al. "Language models are few-shot learners". In: Advances in neural information pr 33 (2020), pp. 1877–1901.

[8] Tim Dettmers et al. "Qlora: Efficient finetuning of quantized llms". In: Advances in neural informa 36 (2023), pp. 10088–10115.

[9] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: Proceedings of the 2019 conference of the North American chapter of 2019, pp. 4171–4186.

[10] Zichuan Fu et al. "Sliding window attention training for efficient large language models". In: arXiv preprint arXiv:2502.18845 (2025).

[11] Daya Guo et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning". In: arXiv preprint arXiv:2501.12948 (2025).

[12] Edward J Hu et al. "Lora: Low-rank adaptation of large language models." In: Iclr 1.2 (2022), p. 3.

[13] Jiedong Lang, Zhehao Guo, and Shuyu Huang. "A comprehensive study on quantization techniques for large language models". In: 2024 4th International conference on artifi IEEE. 2024, pp. 224–231.

[14] Llama 3.1, Model Cards and Prompt formats. URL: https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_1/.

[15] llm-aided-ocr. URL: https://github.com/Dicklesworthstone/llm_aided_ocr.

[16] Shervin Minaee et al. "Large language models: A survey". In: arXiv preprint arXiv:2402.06196 (2024).

[17] Alec Radford et al. "Improving language understanding by generative pre-training". In: (2018).

[18] Oscar Sainz et al. "Gollie: Annotation guidelines improve zero-shot information-extraction". In: arXiv preprint arXiv:2310.03668 (2023).

[19] Erik Tjong Kim Sang and Fien De Meulder. "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition". In: Proceedings of the seventh con 2003, pp. 142–147.

[20] Sander Schulhoff et al. "The prompt report: A systematic survey of prompt engineering techniques". In: arXiv preprint arXiv:2406.06608 (2024).

[21] Wei Liang Seow et al. "A review of named entity recognition: from learning methods to modelling paradigms and tasks". In: Artificial Intelligence Review 58.10 (2025), p. 315.

[22] Noam Shazeer. “Fast transformer decoding: One write-head is all you need”. In: arXiv preprint arXiv:1911.02150 (2019).

[23] Enzo Shiraishi et al. “Retrieval-Enhanced Named Entity Recognition”. In: arXiv preprint arXiv:24 (2024).

[24] Gemma Team et al. “Gemma: Open models based on gemini research and technology”. In: arXiv preprint arXiv:2403.08295 (2024).

[25] Hugo Touvron et al. “Llama: Open and efficient foundation language models”. In: arXiv preprint arXiv:2302.13971 (2023).

[26] Ashish Vaswani et al. “Attention is all you need”. In: Advances in neural information processing sy 30 (2017).

[27] An Vu and Jonas Oppenlaender. “Prompt Engineer: Analyzing skill requirements in the AI job market”. In: arXiv preprint arXiv:2506.00058 (2025).

[28] Shuhe Wang et al. “GPT-NER: Named Entity Recognition via Large Language Models”. In: Findings of the Association for Computational Linguistics: NAACL 2025. Ed. by Luis Chiruzzo, Alan Ritter, and Lu Wang. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 4257–4275. ISBN: 979-8-89176-195-7. DOI: 10.18653/v1/2025.findings-naacl.239. URL: https://aclanthology.org/2025.findings-naacl.239/.

[29] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: Advances in Neural Information Processing Systems. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 24824–24837. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.

[30] Jason Wei et al. “Emergent abilities of large language models”. In: arXiv preprint arXiv:2206.0768
(2022).

[31] Adrian de Wynter. “Is In-Context Learning Learning?” In: arXiv preprint arXiv:2509.10414
(2025).

[32] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. “Hallucination is inevitable: An
innate limitation of large language models”. In: arXiv preprint arXiv:2401.11817
(2024).